计算机系统综合设计A

RISC-V CPU的流水线实现

龚奕利

yiligong@whu.edu.cn

2026年春季

25秋组成-龚奕利班

群号：1004010025

扫一扫二维码，加入群聊

QQ

实验目的

- 融会贯通计算机组成与设计课程所教授的知识，通过对知识的综合应用，加深对CPU系统各模块的工作原理及相互联系的认识。
- 学习采用EDA（Electronic Design Automation）技术设计RISC-V流水线CPU的技术与方法。
- 培养科学研究的独立工作能力，获得CPU设计、仿真、综合、实现、运行的实践和经验。
- 了解SOC系统，并在FPGA开发板上实现简单的SOC系统。

实验内容

- 用硬件描述语言（Verilog）设计 RISC-V **流水线** CPU，支持 RV32I 中的 37 条指令
- 用仿真软件 Modelsim/iVerilog 对有数据冒险和控制冒险的汇编程序进行仿真
- 用 Vivado 下载到 N4 FPGA 板上进行综合、实现和运行

RISC-V指令集

- 4个基本的ISA:
  - RV32I: 32位地址空间整数指令
  - RV64I: 64位地址空间整数指令
  - RV32E: RV32I的子集，更少的整数寄存器，用以支持微小控制器
  - RV128I: 128位地址空间整数指令
- 还有很多extensions
  - Standard, reserved, and custom
  - E.g. the standard integer multiplication and division extension is named “M”
  - The standard atomic instruction extension, denoted by “A”
  - The standard single-precision floating-point extension, denoted by “F”
  - The standard double-precision floating-point extension, denoted by “D”
  - The standard “C” compressed instruction extension provides narrower 16-bit forms of common instructions

待实现的37条指令

- I0={LUI, AUIPC}
- I1={JAL, JALR, BEQ, BNE, BLT, BGE, BLTU, BGEU}
- I2={LB, LH, LW, LBU, LHU, SB, SH, SW}
- I3={ADDI, SLTI, SLTIU, XORI, ANDI, ALLI, SRLI, SRALI, SRAI}
- I4={ADD, SUB, SLL, SLT, SLTU, XOR, SRL, SRA, OR, AND}
- See to The RISC-V Instruction Set Manual: Volume I: Unprivileged ISA “Chapter 24 RV32/64G Instruction Set Listings”
- RISC-V手册（英文、中文）中附录A也有说明

评分标准

条件	分数段	说明
单周期CPU仿真下板成功	60-70	每级评分根据提交的实验报告确定最后分数； 最后阶段实现的功能可以根据班级情况确定。
支持冒险流水线CPU仿真下板成功	70-80	
用RISC_V汇编语言编写应用程序并在自己设计的CPU上运行成功，应用程序要求包含有实现的37条指令	80-90	
实现中断、支持定时器中断、syscall等四种中断处理，实现对VGA和PS2接口驱动，或者其它同等难度的功能	90-100	

参加龙芯杯个人赛，进入决赛，90+

提交方式

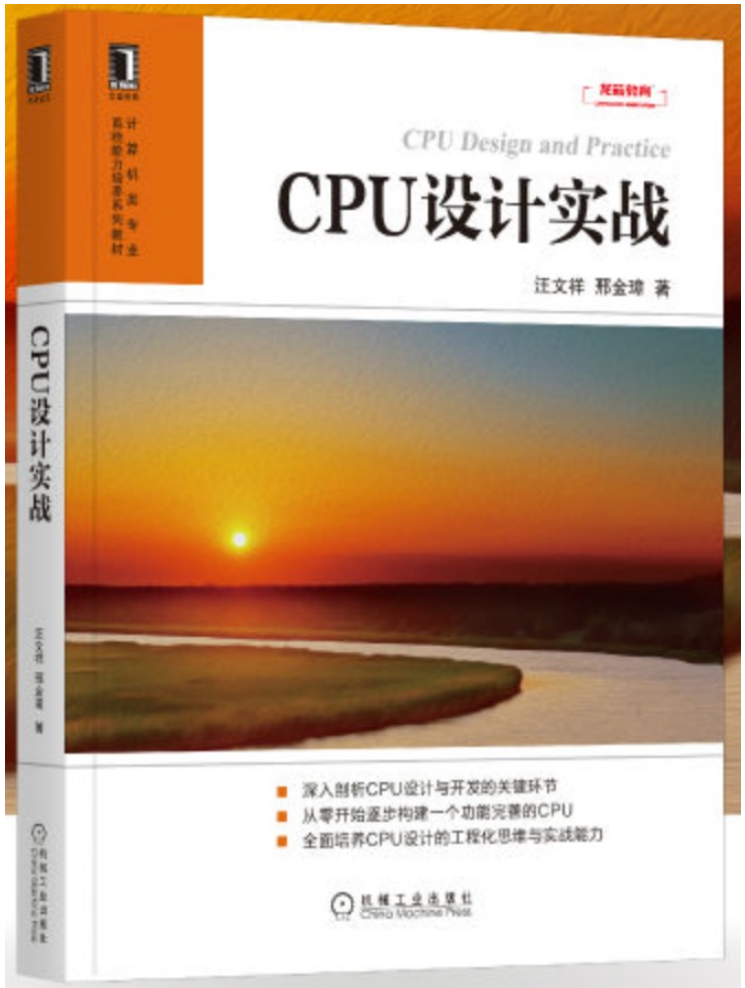
- 代码及实验报告
  - Due on 6月17日 24:00
  - 在https://cslabcg.whu.edu.cn上提交
  - 具体要求和格式稍后给出
- 面试
  - 面试内容包括但不限于代码调试、运行操作、结果展示、现场实现新增指令等
  - 随堂

FPGA的一般设计流程

1) 电路设计：RTL级电路设计方案
2) 代码编写：用HDL（Verilog）表述电路设计方案，成为EDA工具能够看懂的形式
3) 功能仿真：通过软件（ModelSim/iVerilog）仿真模拟的方式查看电路的逻辑功能行为是否符合最初的设计需求
4) 综合实现：HDL代码到真实芯片电路的转换（Vivado）
5) 上板调试：Vivado、 Nexys4-DDR（Xilinx Artix-7）FPGA

关于Verilog

- Tools/Verilog教程
- 计算机原理与设计：Verilog HDL版
  - 是MIPS32 CPU的实现，类似且大量Verilog代码
- CPU设计实战
  - 是MIPS32 CPU的实现，但是更为侧重实际



实验阶段

- 阶段1：搭建五级流水线CPU框架：3月4日
- 阶段2：添加I-type算术逻辑运算指令：
- 阶段3：添加R-type算术逻辑运算指令：3月11日
- 阶段4：添加跳转指令：
- 阶段5：添加访存指令：
- 阶段6：用前递技术解决冲突：
- 阶段7：用冒险检测+停顿解决冲突：4月15日
- 阶段8：基准程序设计、测试与CPU调试：6月10日
- 阶段9：中断、接口等附加功能：6月10日

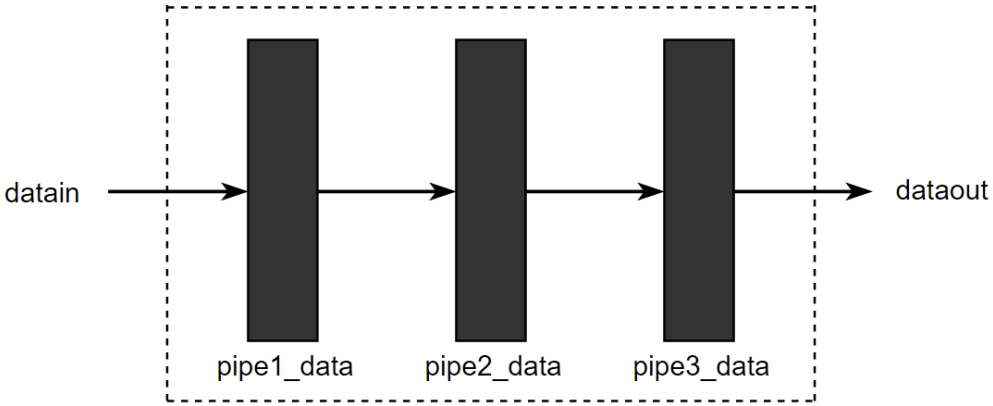
阶段1：搭建五级流水线CPU框架

1) 阅读并理解单周期CPU的代码
   - 建议对照PPT/课本中流水线CPU的框图，对照代码划分出属于IF/ID/EXE/MEM/WB各个阶段的模块
2) 搭建流水线CPU框架
   - 插入流水级寄存器，将单周期CPU拆分为五个阶段
3) 确保原有指令功能无误
   - 梳理CPU各个阶段的逻辑，确保原单周期CPU里已经实现的指令仍能正常运行

流水线CPU的实现

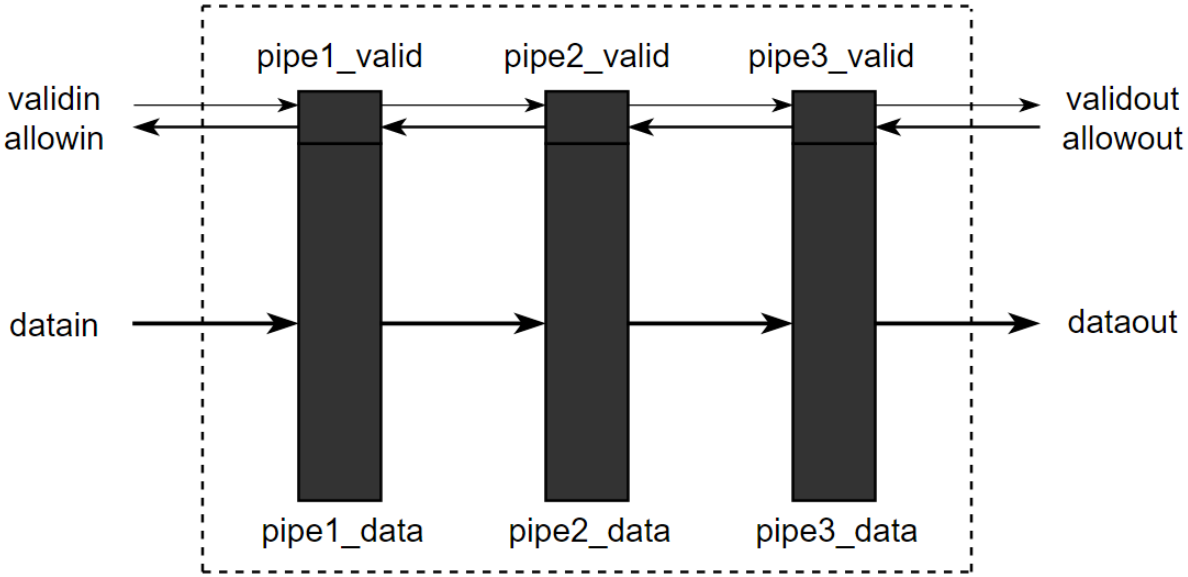
- 无阻塞流水线的实现
- 有阻塞流水线的实现

无阻塞流水线的实现




```
module non_stall_pipeline #
{
    parameter WIDTH = 100
}
{
    input                clk,
    input    [WIDTH-1:0] datain,
    output   [WIDTH-1:0] dataout
};
reg [WIDTH-1:0] pipe1_data;
reg [WIDTH-1:0] pipe2_data;
reg [WIDTH-1:0] pipe3_data;

always @(posedge clk) begin
    pipe1_data <= datain;
end

always @(posedge clk) begin
    pipe2_data <= pipe1_data;
end

always @(posedge clk) begin
    pipe3_data <= pipe2_data;
end

assign dataout = pepe3_data;

endmodule
```

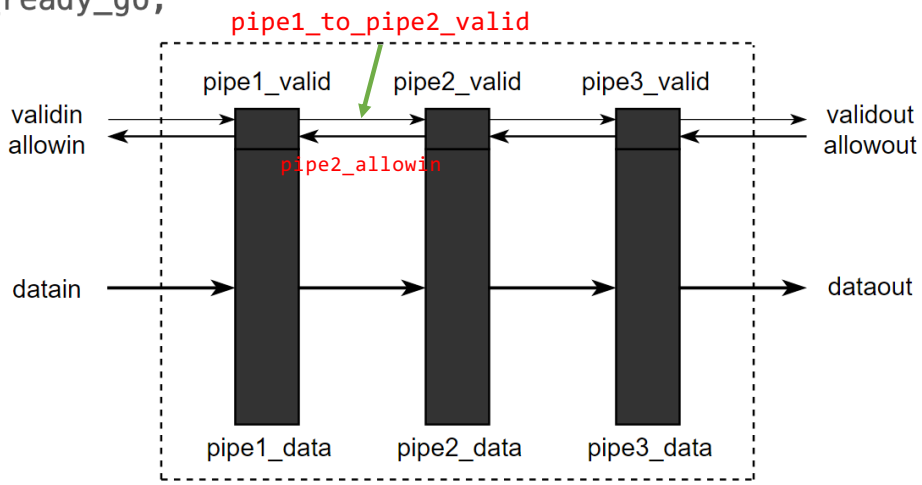
有阻塞流水线的实现




```
module stallable_pipeline #
{
    parameter WIDTH = 100
}
{
    input                clk,
    input                rst,
    input                validin,
    input    [WIDTH-1:0] datain,
    output               validout,
    output   [WIDTH-1:0] dataout
};
reg              pipe1_valid;
reg  [WIDTH-1:0] pipe1_data;
reg              pipe2_valid;
reg  [WIDTH-1:0] pipe2_data;
reg              pipe3_valid;
reg  [WIDTH-1:0] pipe3_data;
```

有阻塞流水线的实现

```
20  //pipeline stage1
21  wire            pipe1_allowin;
22  wire            pipe1_ready_go;
23  wire            pipe1_to_pipe2_valid;
24  assign pipe1_ready_go = ....;
25  assign pipe1_allowin = !pipe1_valid || pipe1_ready_go && pipe2_allowin;
26  assign pipe1_to_pipe2_valid = pipe1_valid && pipe1_ready_go;
27  always @(posedge clk) begin
28      if (rst) begin
29          pipe1_valid <= 1'b0;
30      end
31      else if (pipe1_allowin) begin
32          pipe1_valid <= validin;
33      end
34      if (validin && pipe1_allowin) begin
35          pipe1_data <= datain;
36      end
37  end
```



pipeX_valid：REG 表明流水线级X上当前时钟是否有有效数据

pipeX_ready_go：WIRE 表明流水线级X的任务已经完成，可以传递到第X+1级

pipeX_allowin：WIRE 表明流水线级X可以接收第X-1级送来的数据

pipeX_to_pipeX+1_valid：表明流水线级X有数据需要在下一时钟周期传给第X+1级

有阻塞流水线的实现（续）

```
//pipeline stage2
wire            pipe2_allowin;
wire            pipe2_ready_go;
wire            pipe2_to_pipe3_valid;
assign pipe2_ready_go = ....;
assign pipe2_allowin = !pipe2_valid || pipe2_ready_go && pipe3_allowin;
assign pipe2_to_pipe3_valid = pipe2_valid && pipe2_ready_go;
always @(posedge clk) begin
    if (rst) begin
        pipe2_valid <= 1'b0;
    end
    else if (pipe2_allowin) begin
        pipe2_valid <= pipe1_to_pipe2_valid;
    end
    if (pipe1_to_pipe2_valid && pipe2_allowin) begin
        pipe2_data <= pipe1_data;
    end
end
```

pipeX_to_pipeX+1_valid：表明流水线级X有数据需要在下一时钟周期传给第X+1级

Pipelined Control (Simplified)

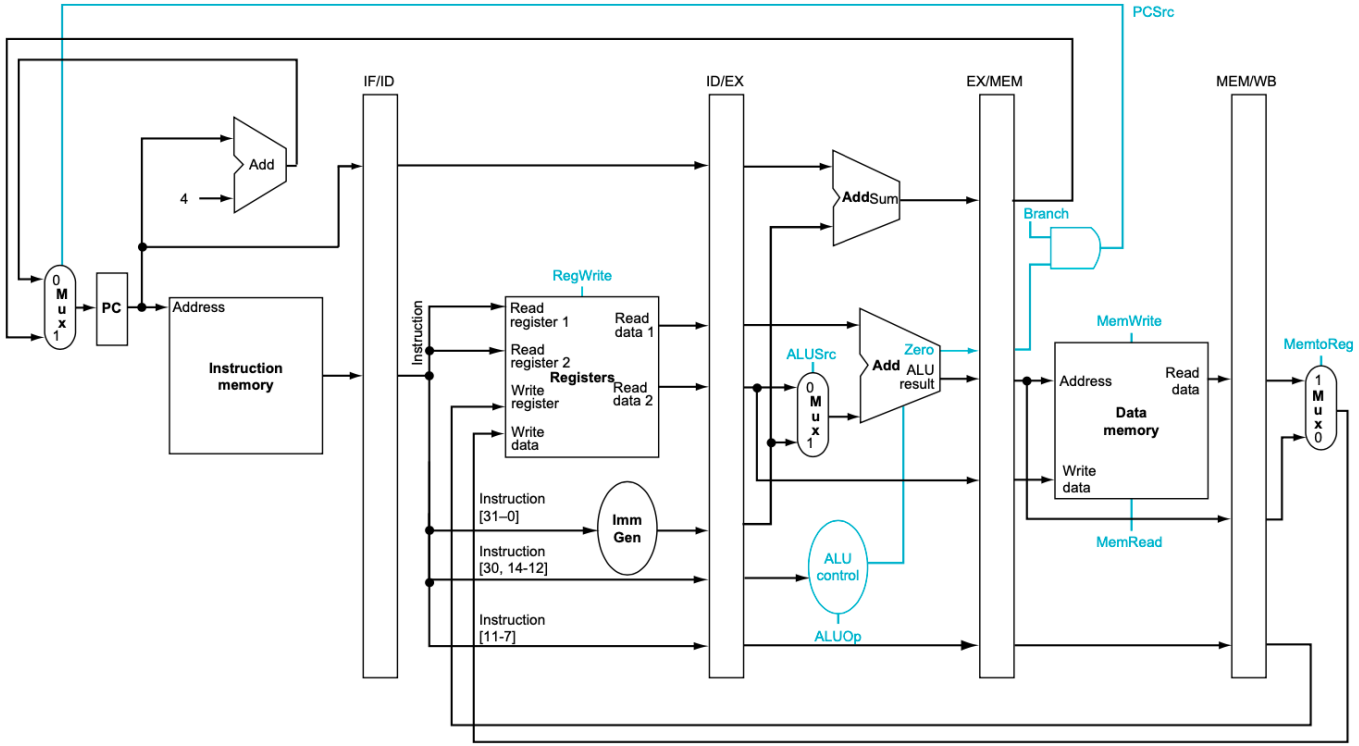
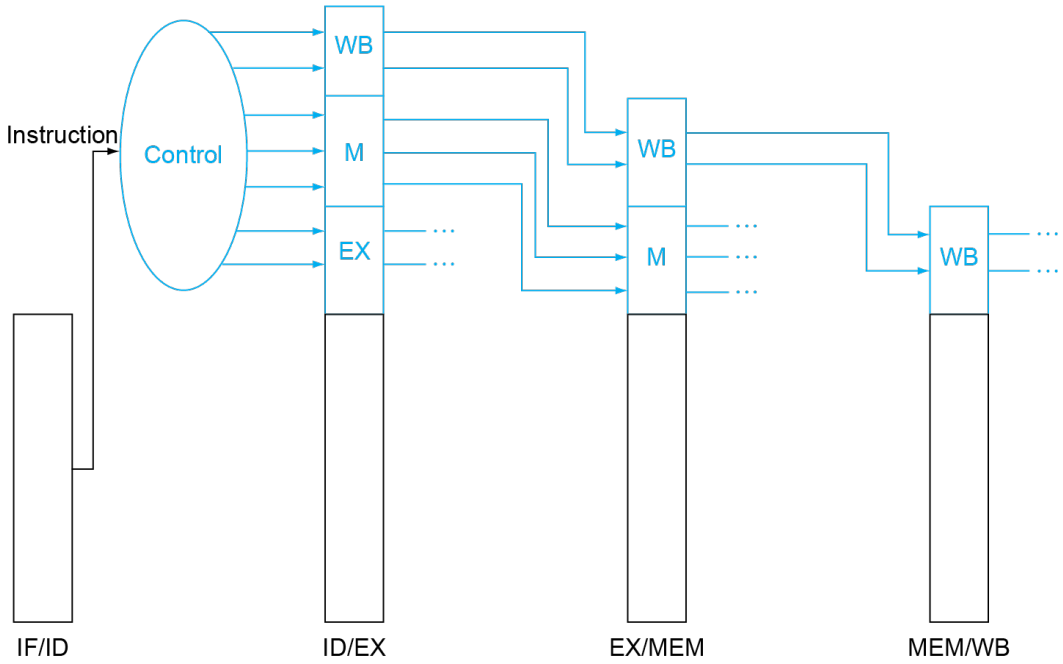


FIGURE 4.48 The pipelined datapath of Figure 4.43 with the control signals identified. This datapath borrows the control logic for PC source, register destination number, and ALU control from Section 4.4. Note that we now need funct fields of the instruction in the EX stage as input to ALU control, so these bits must also be included in the ID/EX pipeline register.

Pipelined Control

- Control signals derived from instruction
  - As in single-cycle implementation



阶段2：添加I-type算术逻辑运算指令

- I0={LUI, AUIPC}
- I3={ADDI, SLTI, SLTIU, XORI, ANDI, ALLI, SRLI, SRALI, SRAI}

imm[31:12]	rd	0110111	LUI
imm[31:12]	rd	0010111	AUIPC

imm[11:0]		rs1	000	rd	0010011	ADDI
imm[11:0]		rs1	010	rd	0010011	SLTI
imm[11:0]		rs1	011	rd	0010011	SLTIU
imm[11:0]		rs1	100	rd	0010011	XORI
imm[11:0]		rs1	110	rd	0010011	ORI
imm[11:0]		rs1	111	rd	0010011	ANDI
0000000	shamt	rs1	001	rd	0010011	SLLI
0000000	shamt	rs1	101	rd	0010011	SRLI
0100000	shamt	rs1	101	rd	0010011	SRAI

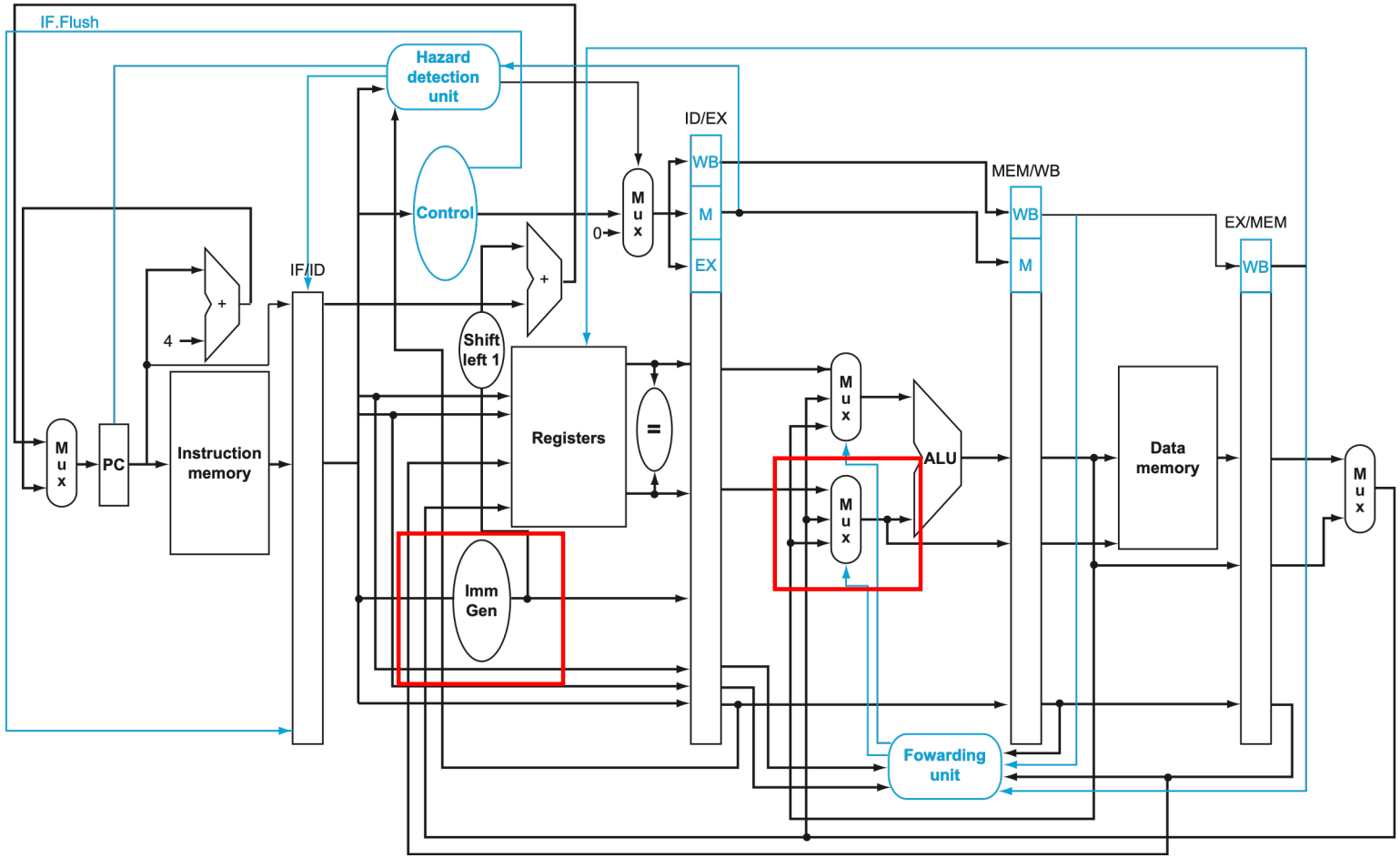


FIGURE 4.62 The final datapath and control for this chapter.
Note that this is a stylized figure rather than a detailed datapath, so it's missing the ALUsrc Mux from Figure 4.55 and the multiplexor controls from Figure 4.49.

阶段3：添加R-type算术逻辑运算指令

- I4={ADD, SUB, SLL, SLT, SLTU, XOR, SRL, SRA, OR, AND}

0000000	shamt	rs1	001	rd	0010011	SLLI
0000000	shamt	rs1	101	rd	0010011	SRLI
0100000	shamt	rs1	101	rd	0010011	SRAI
0000000	rs2	rs1	000	rd	0110011	ADD
0100000	rs2	rs1	000	rd	0110011	SUB
0000000	rs2	rs1	001	rd	0110011	SLL
0000000	rs2	rs1	010	rd	0110011	SLT
0000000	rs2	rs1	011	rd	0110011	SLTU
0000000	rs2	rs1	100	rd	0110011	XOR
0000000	rs2	rs1	101	rd	0110011	SRL
0100000	rs2	rs1	101	rd	0110011	SRA
0000000	rs2	rs1	110	rd	0110011	OR
0000000	rs2	rs1	111	rd	0110011	AND

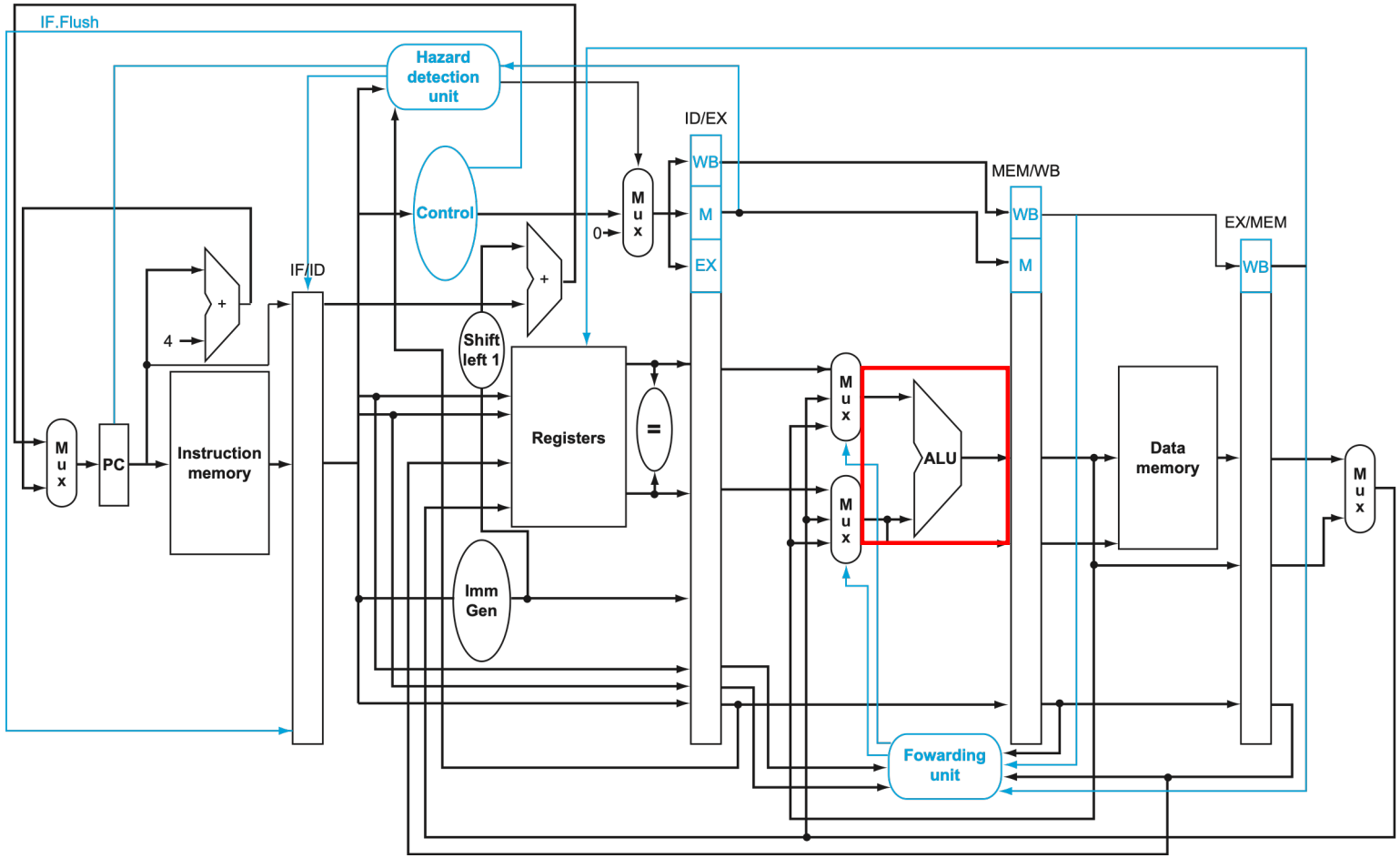


FIGURE 4.62 The final datapath and control for this chapter.
Note that this is a stylized figure rather than a detailed datapath, so it's missing the ALUsrc Mux from Figure 4.55 and the multiplexor controls from Figure 4.49.

阶段4：添加跳转指令

- I1={JAL, JALR, BEQ, BNE, BLT, BGE, BLTU, BGEU}

imm[20|10:1|11|19:12]				rd	1101111	JAL
imm[11:0]		rs1	000	rd	1100111	JALR
imm[12|10:5]	rs2	rs1	000	imm[4:1|11]	1100011	BEQ
imm[12|10:5]	rs2	rs1	001	imm[4:1|11]	1100011	BNE
imm[12|10:5]	rs2	rs1	100	imm[4:1|11]	1100011	BLT
imm[12|10:5]	rs2	rs1	101	imm[4:1|11]	1100011	BGE
imm[12|10:5]	rs2	rs1	110	imm[4:1|11]	1100011	BLTU
imm[12|10:5]	rs2	rs1	111	imm[4:1|11]	1100011	BGEU

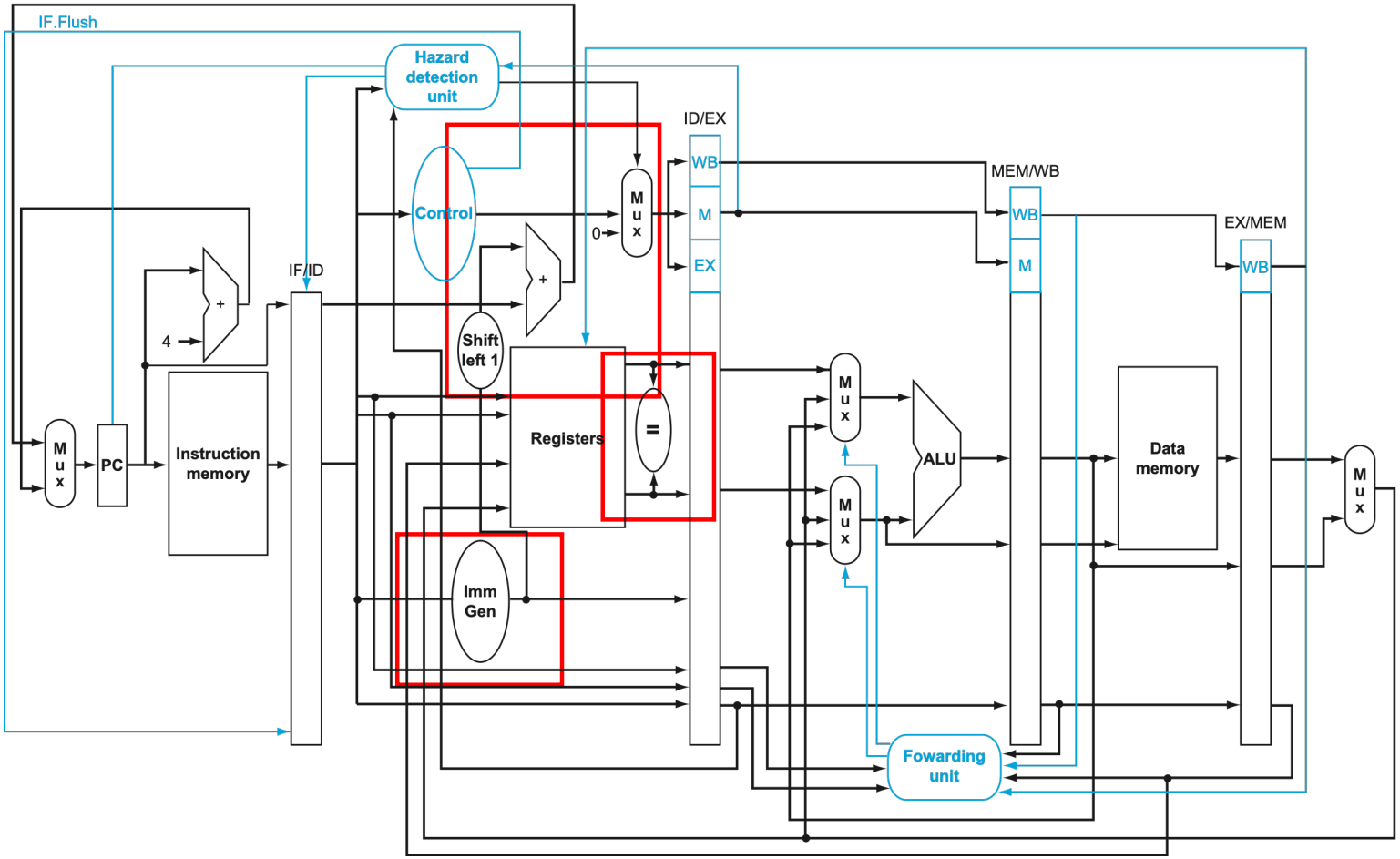


FIGURE 4.62 The final datapath and control for this chapter.
Note that this is a stylized figure rather than a detailed datapath, so it's missing the ALUsrc Mux from Figure 4.55 and the multiplexor controls from Figure 4.49.

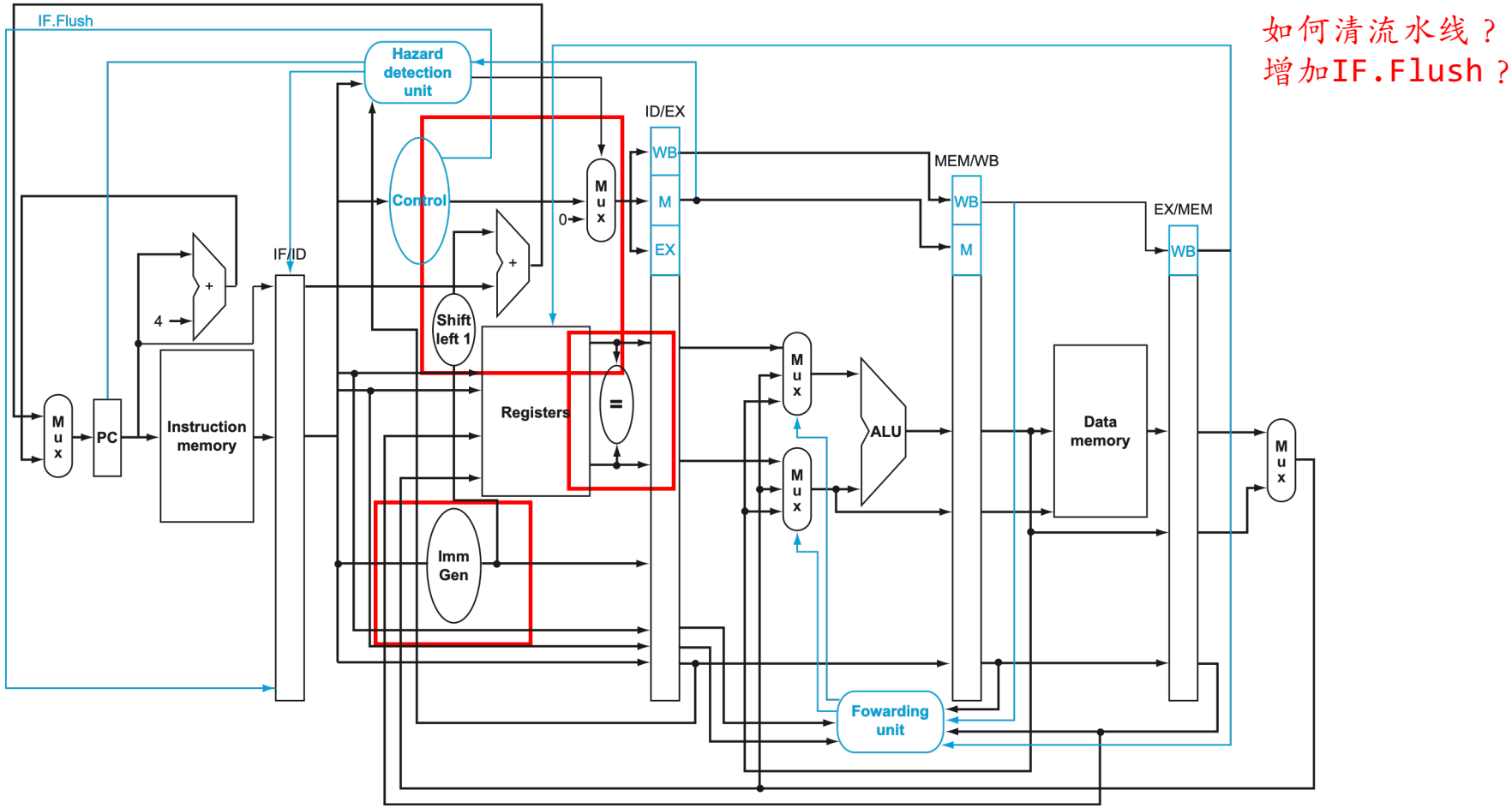


FIGURE 4.62 The final datapath and control for this chapter.
Note that this is a stylized figure rather than a detailed datapath, so it's missing the ALUsrc Mux from Figure 4.55 and the multiplexor controls from Figure 4.49.

阶段5：添加访存指令

- I2={LB, LH, LW, LBU, LHU, SB, SH, SW}

imm[11:0]		rs1	000	rd	0000011	LB
imm[11:0]		rs1	001	rd	0000011	LH
imm[11:0]		rs1	010	rd	0000011	LW
imm[11:0]		rs1	100	rd	0000011	LBU
imm[11:0]		rs1	101	rd	0000011	LHU
imm[11:5]	rs2	rs1	000	imm[4:0]	0100011	SB
imm[11:5]	rs2	rs1	001	imm[4:0]	0100011	SH
imm[11:5]	rs2	rs1	010	imm[4:0]	0100011	SW

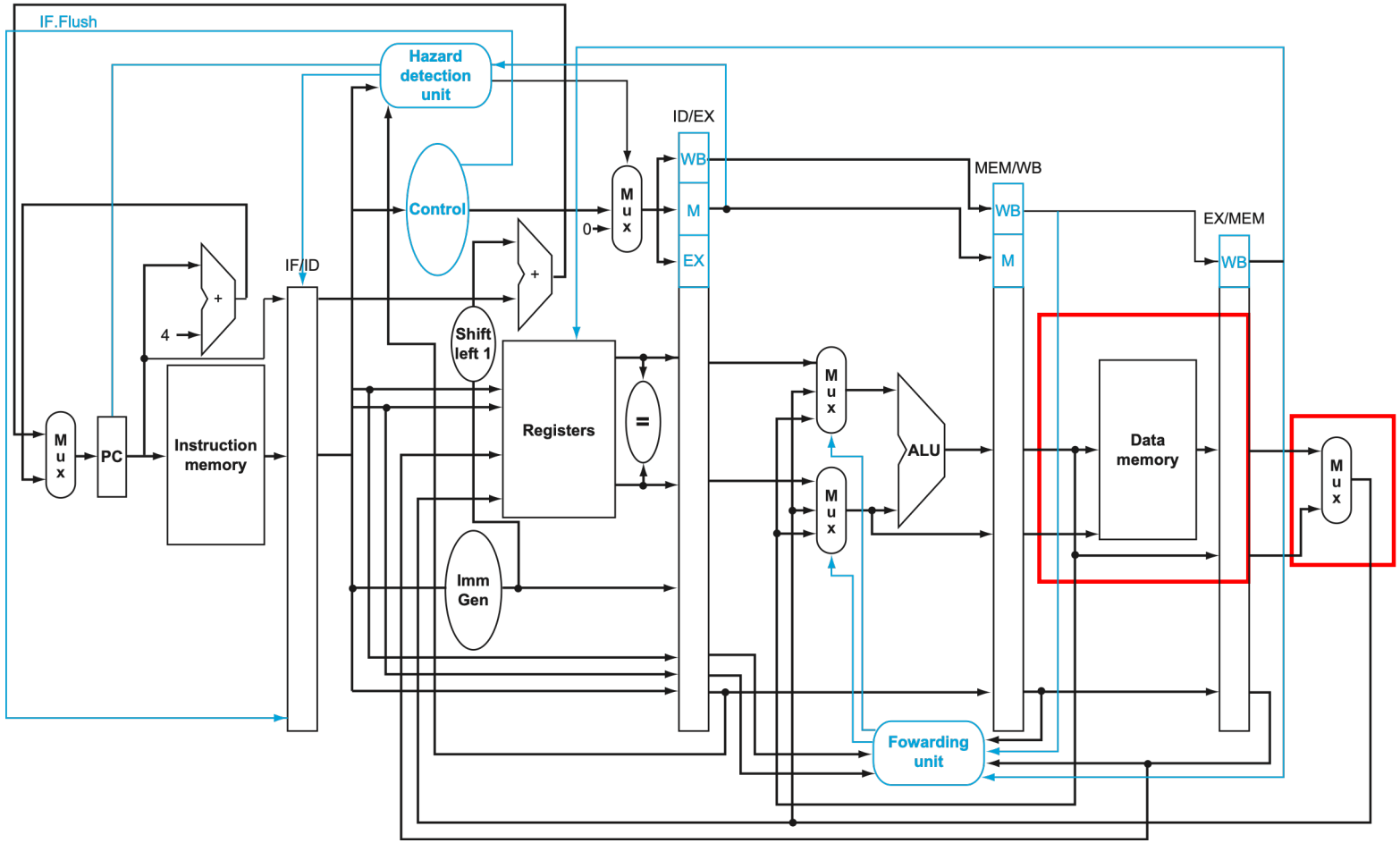
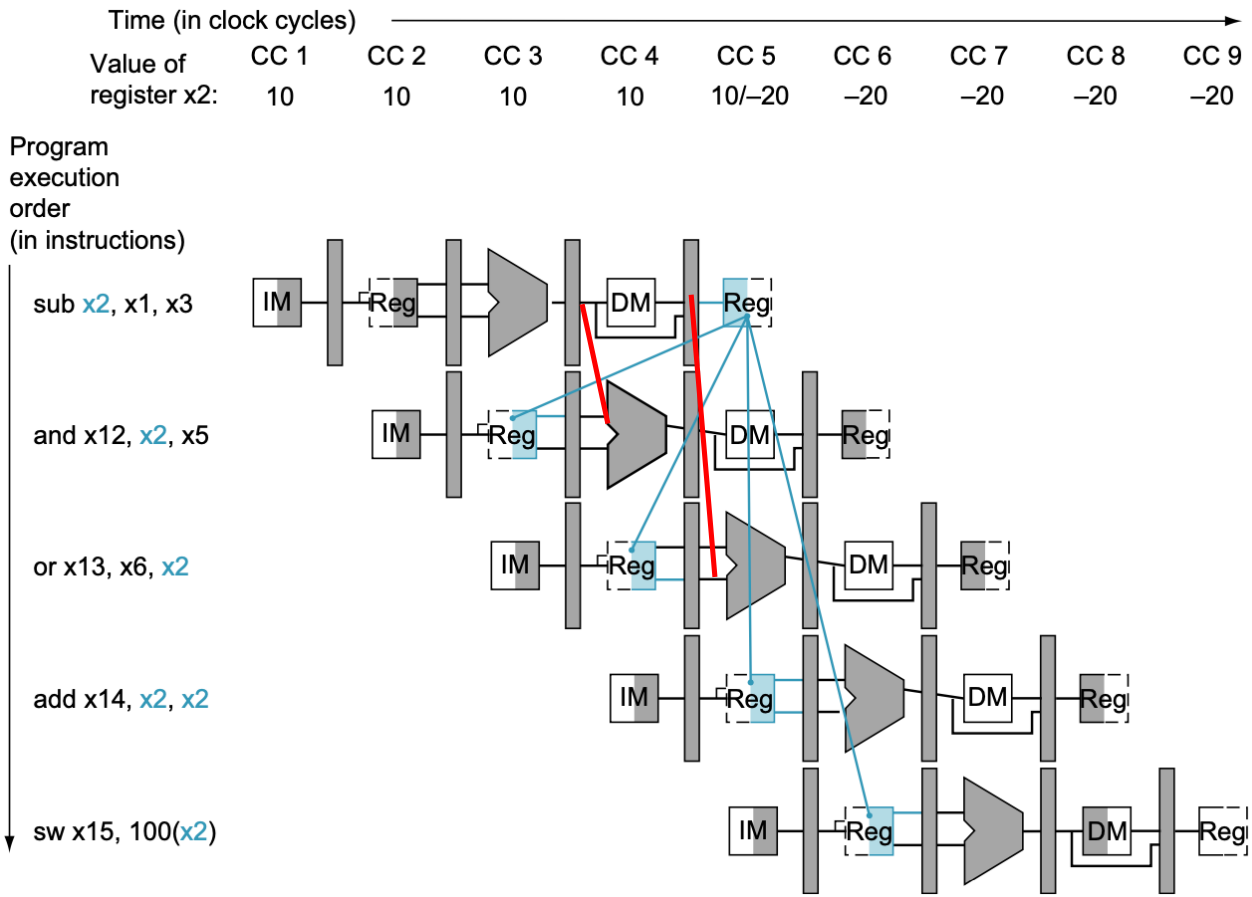


FIGURE 4.62 The final datapath and control for this chapter.
Note that this is a stylized figure rather than a detailed datapath, so it's missing the ALUsrc Mux from Figure 4.55 and the multiplexor controls from Figure 4.49.

阶段6：用前递技术解决冲突



阶段6：用前递技术解决冲突

- Forwarding Condition: EX/MEM hazard
  - if (EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
    and (EX/MEM.RegisterRd = ID/EX.RegisterRs1)) ForwardA = 01
  - if (EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
    and (EX/MEM.RegisterRd = ID/EX.RegisterRs2)) ForwardB = 01

Double Data Hazard

- Consider the sequence:

```
add x1,x1,x2
add x1,x1,x3
add x1,x1,x4
```

- Both hazards occur
  - Want to use the most recent
- Revise MEM hazard condition
  - Only fwd if EX hazard condition isn’t true

阶段6：用前递技术解决冲突

- Forwarding Condition: MEM/WB hazard
  - if (MEM/WB.RegWrite and (MEM/WB.RegisterRd ≠ 0)
    and not(EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
    and (EX/MEM.RegisterRd = ID/EX.RegisterRs1))
    and (MEM/WB.RegisterRd = ID/EX.RegisterRs1)) ForwardA = 01
  - if (MEM/WB.RegWrite and (MEM/WB.RegisterRd ≠ 0)
    and not(EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
    and (EX/MEM.RegisterRd = ID/EX.RegisterRs2))
    and (MEM/WB.RegisterRd = ID/EX.RegisterRs2)) ForwardB = 01

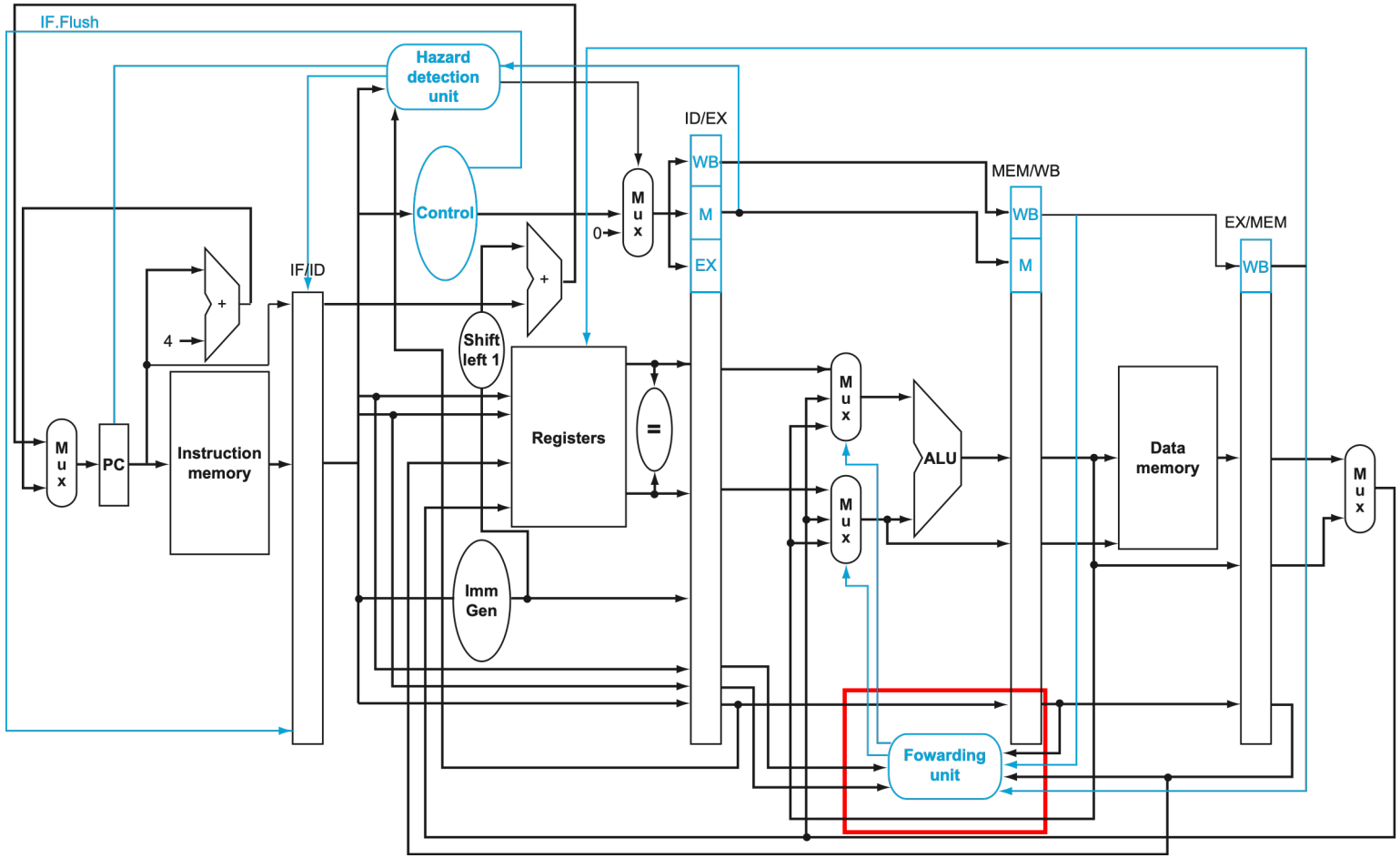


FIGURE 4.62 The final datapath and control for this chapter.
Note that this is a stylized figure rather than a detailed datapath, so it's missing the ALUsrc Mux from Figure 4.55 and the multiplexor controls from Figure 4.49.

阶段6：用前递技术解决冲突

- 分支指令在 ID 阶段执行

```
add     x10, x8, x9
beq     x10, x11, else
```

怎么办？

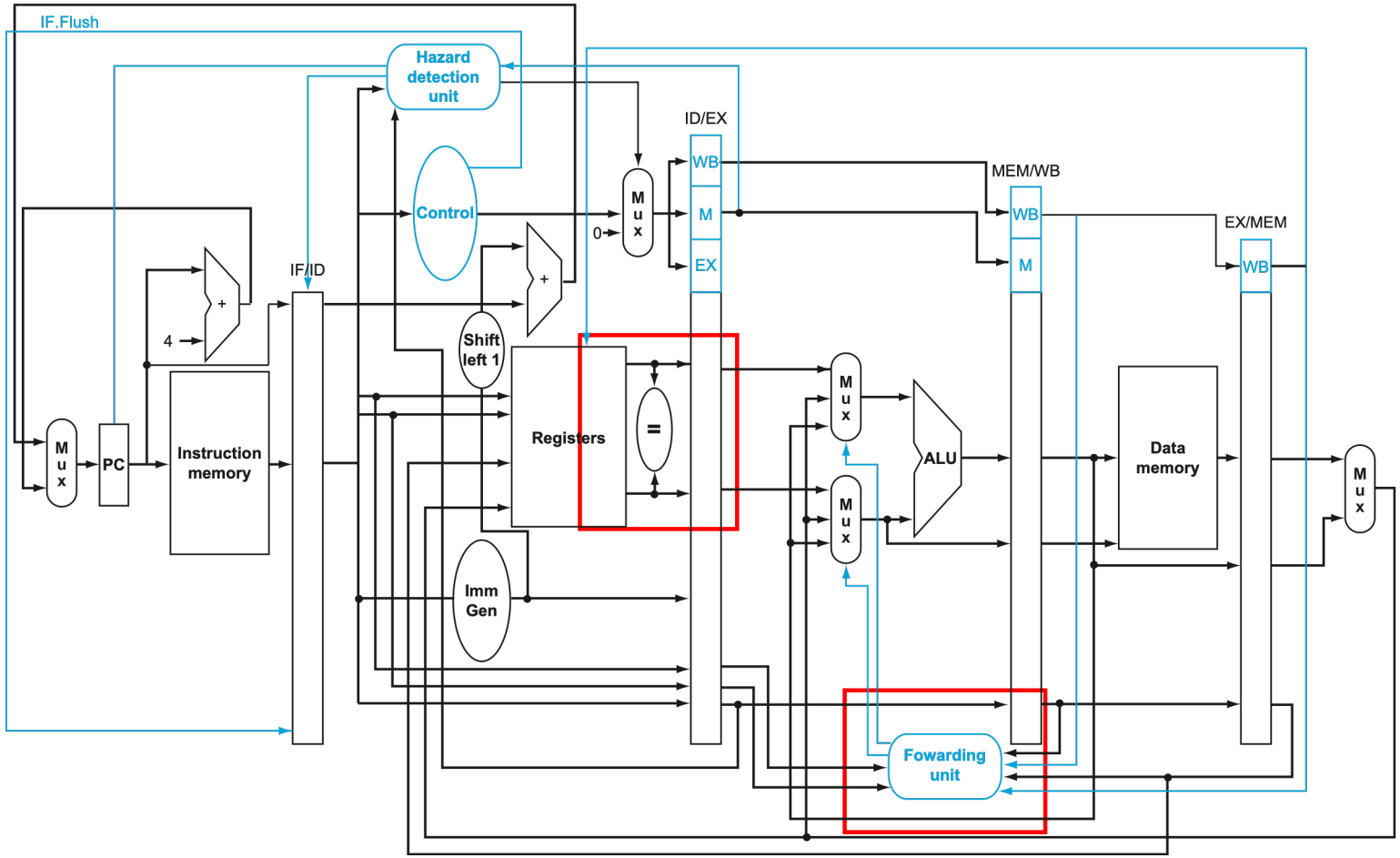


FIGURE 4.62 The final datapath and control for this chapter.
Note that this is a stylized figure rather than a detailed datapath, so it's missing the ALUsrc Mux from Figure 4.55 and the multiplexor controls from Figure 4.49.

阶段7：用冒险检测+停顿解决冲突

- Condition: Load-use hazard
  - ID/EX.MemRead
    and ((ID/EX.RegisterRd = IF/ID.RegisterRs1)
    or (ID/EX.RegisterRd = IF/ID.RegisterRs1))

阶段7：用冒险检测+停顿解决冲突

- Condition: Load-use hazard
  - ID/EX.MemRead
    and ((ID/EX.RegisterRd = IF/ID.RegisterRs1)
    or (ID/EX.RegisterRd = IF/ID.RegisterRs1))

```
ld      x10, 0(x8)
beq     x10, x11, else
```

怎么办？

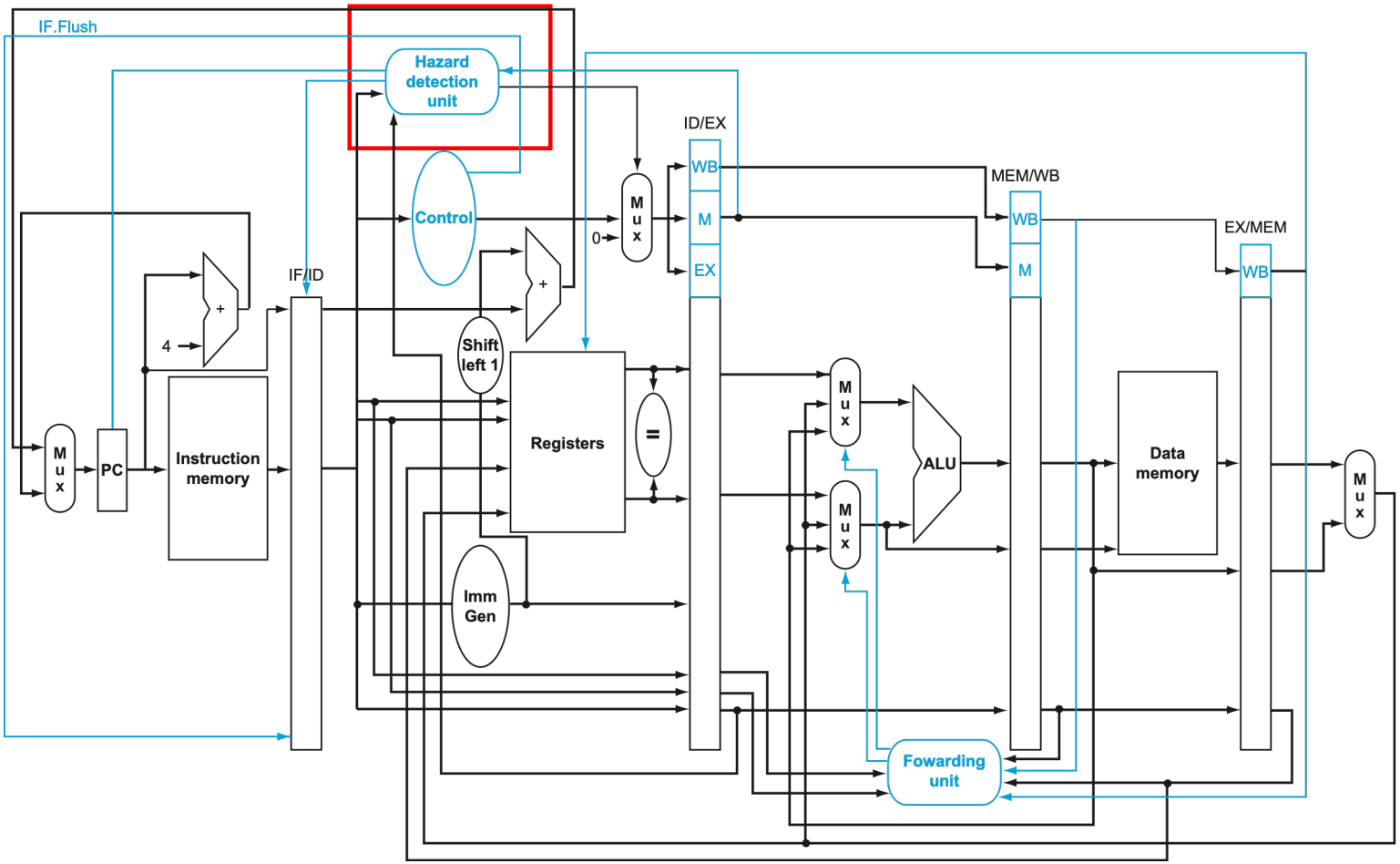


FIGURE 4.62 The final datapath and control for this chapter.
Note that this is a stylized figure rather than a detailed datapath, so it's missing the ALUsrc Mux from Figure 4.55 and the multiplexor controls from Figure 4.49.

阶段8：基准程序设计及测试、调试

- RISC-V编译器
  - Venus: https://venus.cs61c.org/
    https://venus.kvakil.me/
  - RARS: https://github.com/TheThirdOne/rars

  - 汇编代码→RISC-V机器代码，仿真执行

阶段9：中断及课程面试

- 中断机制
- 输入输出系统
  - 键盘：https://nju-projectn.github.io/dlco-lecture-note/exp/07.html
  - VGA接口控制器：https://nju-projectn.github.io/dlco-lecture-note/exp/08.html
  - 字符输入界面：https://nju-projectn.github.io/dlco-lecture-note/exp/09.html
- 或其他自定义内容
- 选择一个实现即可

工具使用：ModelSim

- ModelSim教程
  - modelsim详细使用教程.pdf
  - modelsim_tutorial_v10.1c.pdf
- 项目导入/打开
  - ModelSim使用的是绝对路径，而不是相对路径。如果想直接使用提供的源文件，需要点击File->Change Directory，更改路径到你的目录位置，再打开工程文件
  - 推荐重新创建工程文件，减少不必要的麻烦
- 参见附录一

工具使用：iVerilog + gtkwave

- 安装及使用教程
- 测试用例
- 见共享文件中的iverilog目录

基于iVerilog+GTKwave的轻量级仿真环境

Icarus Verilog (iverilog) 是一个轻量、开源、免费的 Verilog 编译器，支持全平台：Windows/Linux/MacOS。可以使用命令行编译 Verilog 文件，生成对应的仿真波形文件，并通过自带的 gtkwave 查看仿真波形。

安装

Windows

https://bleyer.org/icarus/ ，直接选择最新版本下载即可（应该已经整合了 gtkwave）。

Linux

运行以下两条指令即可：

```
sudo apt-get install iverilog
sudo apt-get install gtkwave
```

MacOS

```
brew install icarus-verilog
brew install gtkwave
```

注意：如果是MACOS 14，安装gtkwave后不能正常打开，先卸载之前的版本，再重新安装。

```
brew uninstall --cask gtkwave
brew install --HEAD randomplum/gtkwave/gtkwave
```

安装后，运行命令：

```
iverilog
```

若安装成功，运行命令后应出现类似如下的界面：

```
conjee@Yilis-MacBook-Pro Downloads % iverilog
iverilog: no source files.

Usage: iverilog [-EiRSuvV] [-B base] [-c cmdfile|-f cmdfile]
                [-g1995|-g2001|-g2005|-g2005-sv|-g2009|-g2012] [-g<feature>]
                [-D macro[=defn]] [-I includedir] [-L moduledir]
                [-M [mode=]depfile] [-m module]
                [-N file] [-o filename] [-p flag=value]
                [-s topmodule] [-t target] [-T min|typ|max]
                [-W class] [-y dir] [-Y suf] [-l file] source_file(s)

See the man page for details.
```

示例

我们以对一个 ALU 元件进行仿真为例，演示调试的基本流程。

首先，用任何你喜欢的文本编辑器编辑如下allu.v文件。

```
module alu(A,B,ALUOp,C);
    input [31:0] A,B;
    input [2:0] ALUOp;
```

代码目录

- riscv测试代码

有数据依赖测试代码

FPGA测试代码

无数据依赖测试代码

prep-machine-code.py

prep-coe-code.py

prep-machine-code.py

riscv32_no_data_dependence_sim_list.txt

riscv32_sim4.asm

riscv32_sim3.asm

riscv32_sim2.asm

riscv32_sim5.asm

riscv32_sim1.asm

riscv32_sim6.asm

fibonacci.asm

riscv-studentnosorting.asm

从仿真代码到FPGA代码

1. 删除module imem的定义，后面会用系统提供的IP实现imem
2. 删除module xgriscv_pipeline的定义
3. 修改fpga_top.v和你的cpu实现：
   1. 在你的cpu文件中添加这几行：
      - Reg_sel：数码管显示寄存器编号
      - Reg_data：数码管显示寄存器数据
   2. 在fpga_top.v文件里把cpu模块名替换成你的cpu模块名

Project Summary × sccpu.v ×

```
10    output        MemWrite;  // memory write
11    output [31:0] aluout;    // ALU output
12    output [31:0] writedata; // data to data memory
13
14    input  [4:0] reg_sel;    // register selection (for debug use)
15    output [31:0] reg_data;  // selected register data (for debug use)
16
17    wire        RegWrite;    // control signal to register write
18    wire        EXTOp;       // control signal to signed extension
19    wire [2:0]  ALUOp;       // ALU opertion
20    wire [1:0]  NPCOp;       // next PC operation
21
```

从仿真代码到FPGA代码

Project Summary × sccpu.v ×

```
10    output        MemWrite;   // memory write
11    output [31:0] aluout;     // ALU output
12    output [31:0] writedata;  // data to data memory
13
14    input  [4:0] reg_sel;     // register selection (for debug use)
15    output [31:0] reg_data;   // selected register data (for debug use)
16
17    wire        RegWrite;     // control signal to register write
18    wire        EXTOp;        // control signal to signed extension
19    wire [2:0]  ALUOp;        // ALU opertion
20    wire [1:0]  NPCOp;        // next PC operation
21
```

在你的cpu文件中添加这几行：
Reg_sel：数码管显示寄存器编号
Reg_data：数码管显示寄存器数据

在fpga_top.v文件里把cpu模块名替换成你的cpu模块名

FPGA版Verilog描述中的注意事项

- 代码中禁止出现initial语句
- 代码中禁止出现casex、casez
- 代码中禁止出现用“#”表达电路延迟
- 时钟信号clock只允许出现在always @ (posedge clock)语句中

注意事项

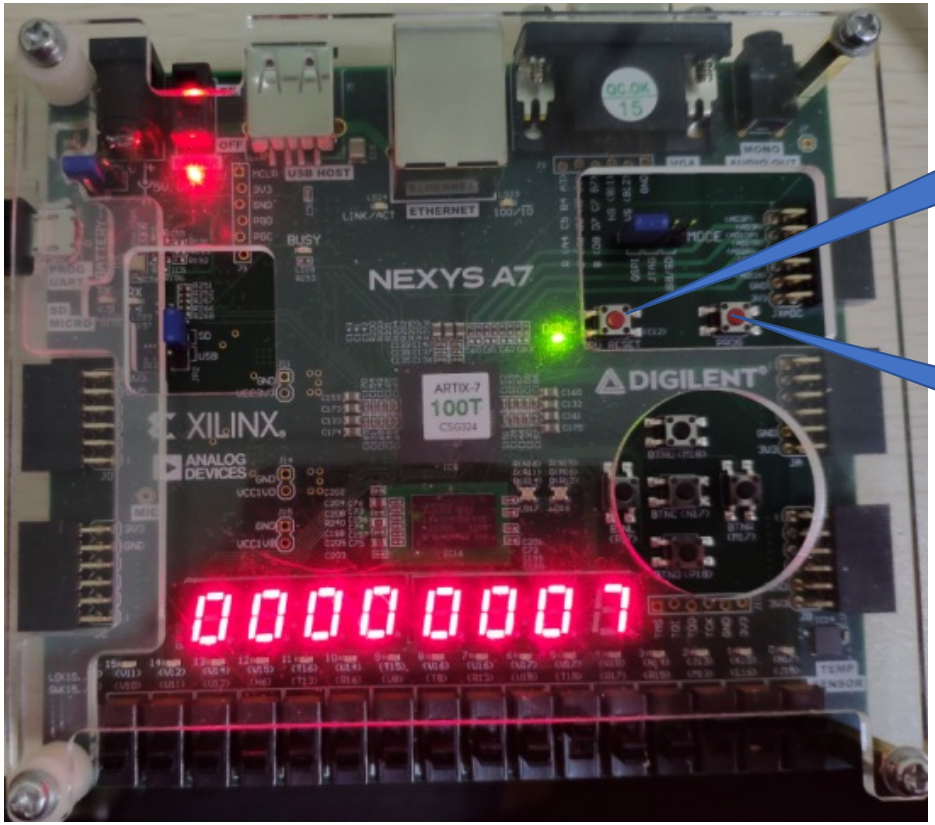
- 没有单步的功能
- 但是可以慢速运行
- 也可以运行完后再看寄存器的值
- 需记录下实验中遇到的问题是怎么解决的，并体现在实验报告中

附录一给出了 Vivado 的基本使用流程

如果到这里都没有错误，恭喜你，你的设计已经成功地下载到FPGA板上运行起来了！

接下来，了解一下如何查看结果吧！

运行结果检查



CPU复位按钮，按下后PC从0开始执行程序

开发板复位按钮，按下后加载Out of Box Demo

- SW5-SW0为0时，七段数码管显示CPU数据；
- 拨动开关(SW15-SW6)，16个开关的状态被写入右边四个七段数码管（左边四个为0）
  - 例如当SW15，SW14，SW7为1，其他开关为0时，七段数码管上显示为0000C080

- SW5为1表示查看寄存器的值
- SW4-SW0为寄存器编号，当SW1为1时，其他为0时，表示R2
- 其他开关均为0
- R2 = 7
- 将SW15置1，CPU慢速运行；按下CPU复位按钮，程序从0地址开始执行，可以看到R2的值开始为5，后来变为7

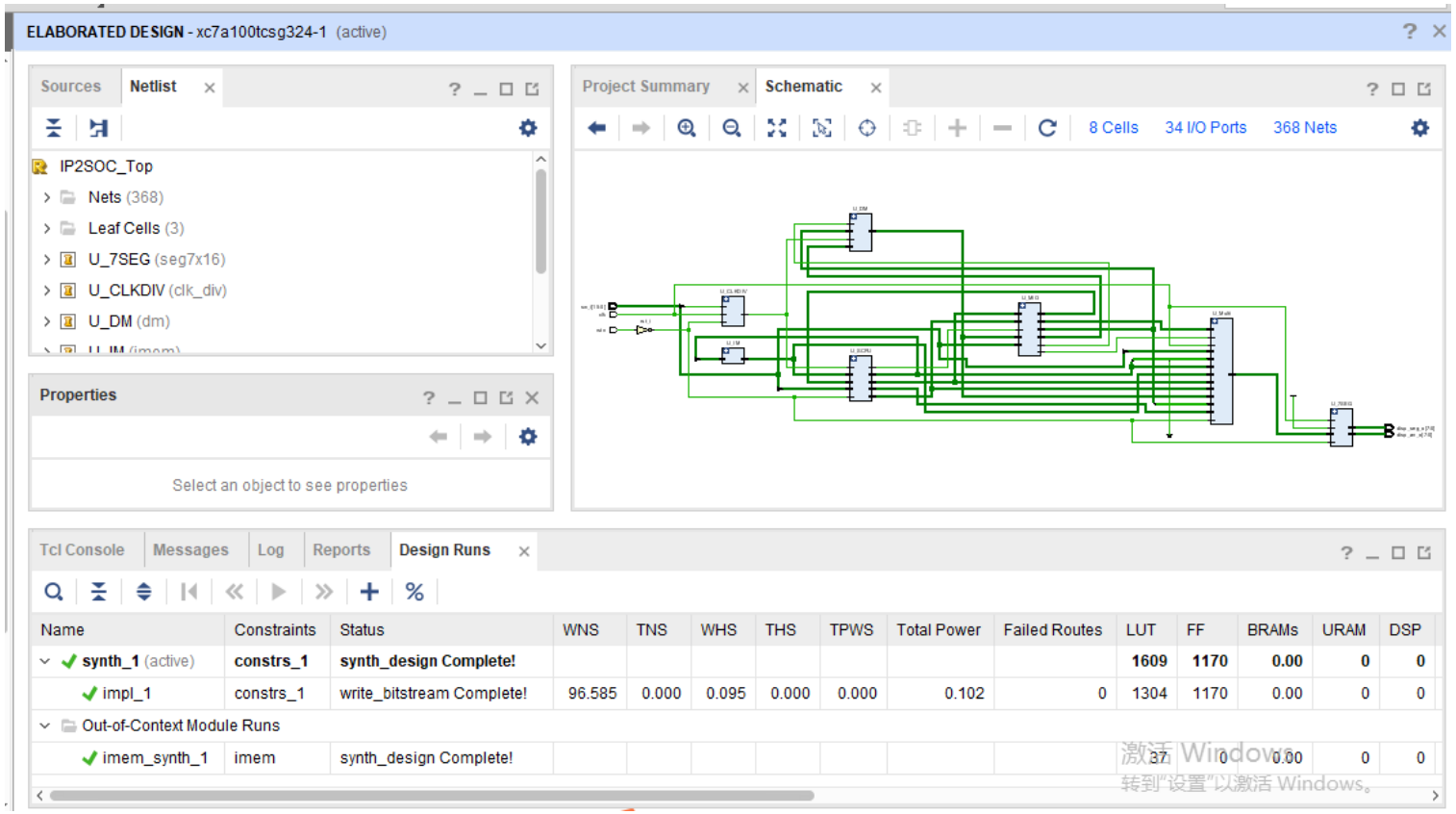
测试开关设置

- 开关作为输入(lw)，对应地址为 0xFFFF0004,
- 七段数码管作为输出（sw），对应地址为 0xFFFF000C

开关设置						功能
SW15	0					CPU全速运行
	1					CPU慢速运行
SW5	SW4	SW3	SW2	SW1	SW0	七段数码管的内容
0	0	0	0	0	0	CPU数据（指令中写入七段数码管的数据，即sw指令写入地址0xFFFF000C的值）；如果代码中没有该指令，则显示默认值0xAA5555AA
0	0	0	0	0	1	指令编号（PC>>2）
0	0	0	0	1	0	指令地址（PC）
0	0	0	0	1	1	指令（IR）
0	0	0	1	0	0	CPU访问存储器或外设的地址
0	0	0	1	0	1	CPU写入存储器或外设的数据
0	0	0	1	1	0	数据存储器读出的数据
0	0	0	1	1	1	数据存储器的访问地址
0	x	1	x	x	x	0xFFFFFFFF
0	1	x	x	x	x	0xFFFFFFFF
1	寄存器的编号					对应寄存器的值

左侧Flow Navigator窗口中，RTL Analysis->Open Elaborated Design

可以看到项目的顶层设计，右击每个方块可以View Source，View Definition

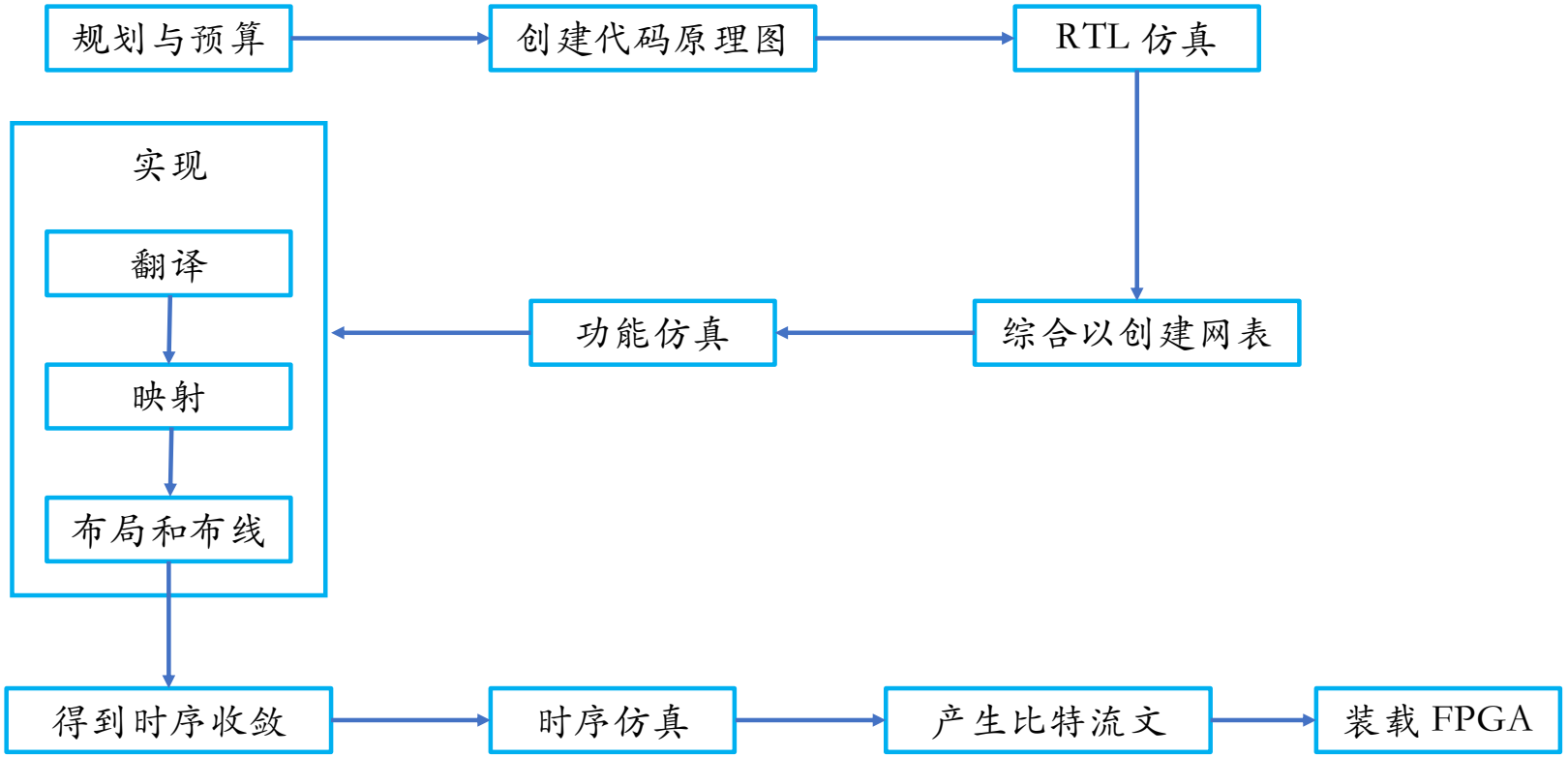


Have fun!

附录一

Vivado 的使用

FPGA 开发过程


规划与预算
创建代码原理图
RTL 仿真
实现
翻译
映射
布局和布线
功能仿真
综合以创建网表
得到时序收敛
时序仿真
产生比特流文
装载 FPGA

Vivado软件

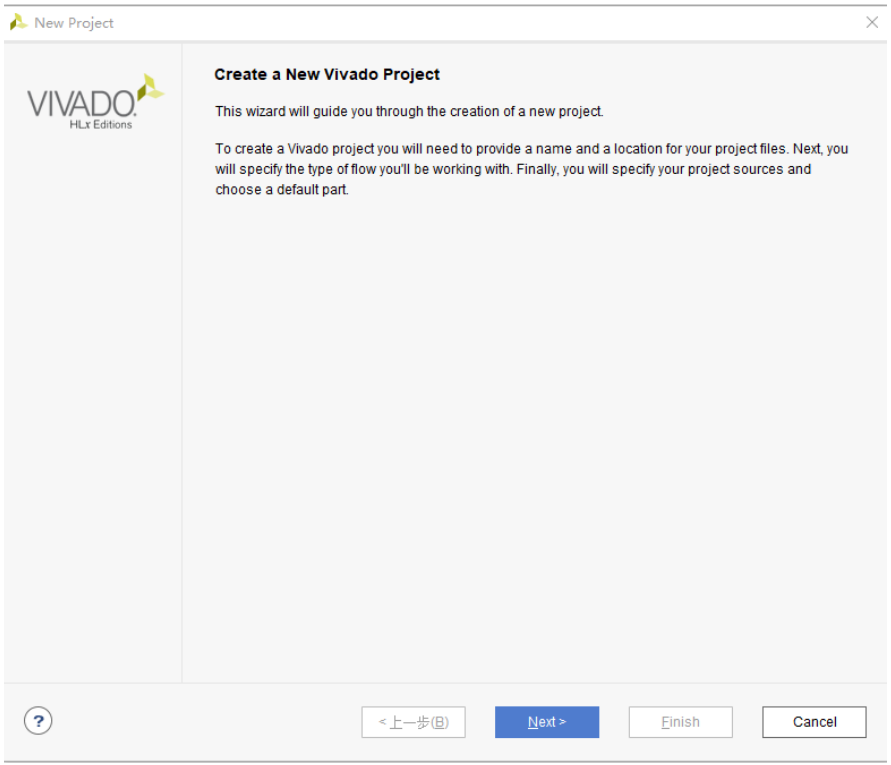
- Vivado设计套件，是FPGA厂商赛灵思（Xilinx）公司2012年发布的集成设计环境。
- 包括高度集成的设计环境和新一代从系统到IC级的工具，这些均建立在共享的可扩展数据模型和通用调试环境基础上。
- 基于AMBA AXI4 互联规范、IP-XACT IP封装元数据、工具命令语言(TCL)、Synopsys 系统约束(SDC) 以及其它有助于根据客户需求量身定制设计流程并符合业界标准的开放式环境。
- 赛灵思构建的Vivado 工具把各类可编程技术结合在一起，能够扩展多达1 亿个等效ASIC门的设计。

Vivado的安装

- 安装路径不能包含中文
- 需要注册Xilinx账号，按照提示要求去做即可
- 安装过程中，选择Vivado HL Design Edition
- 可以参考《CPU设计实战》附录B

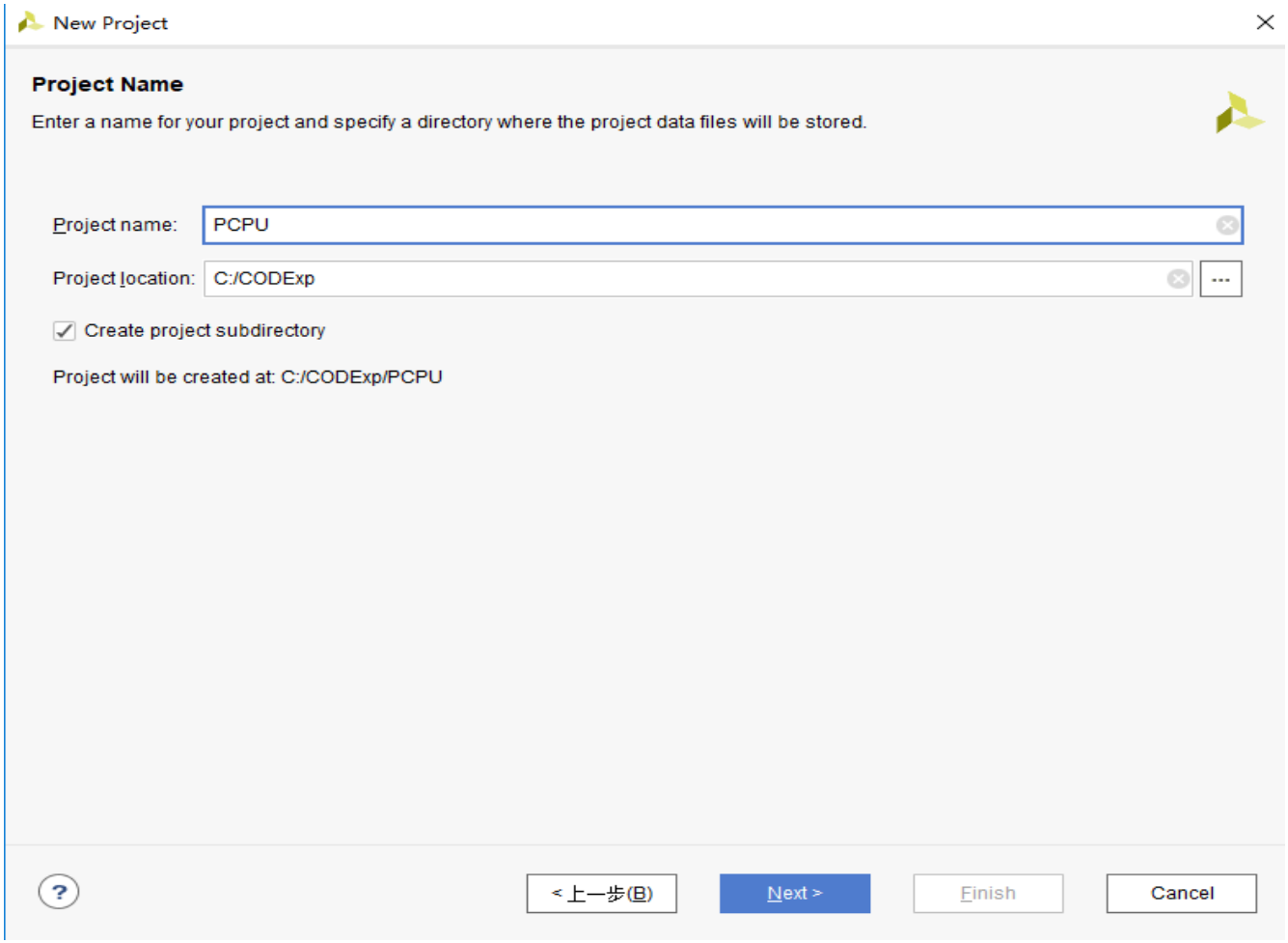
新建工程

- 在Vivado主界面中，File->New Project

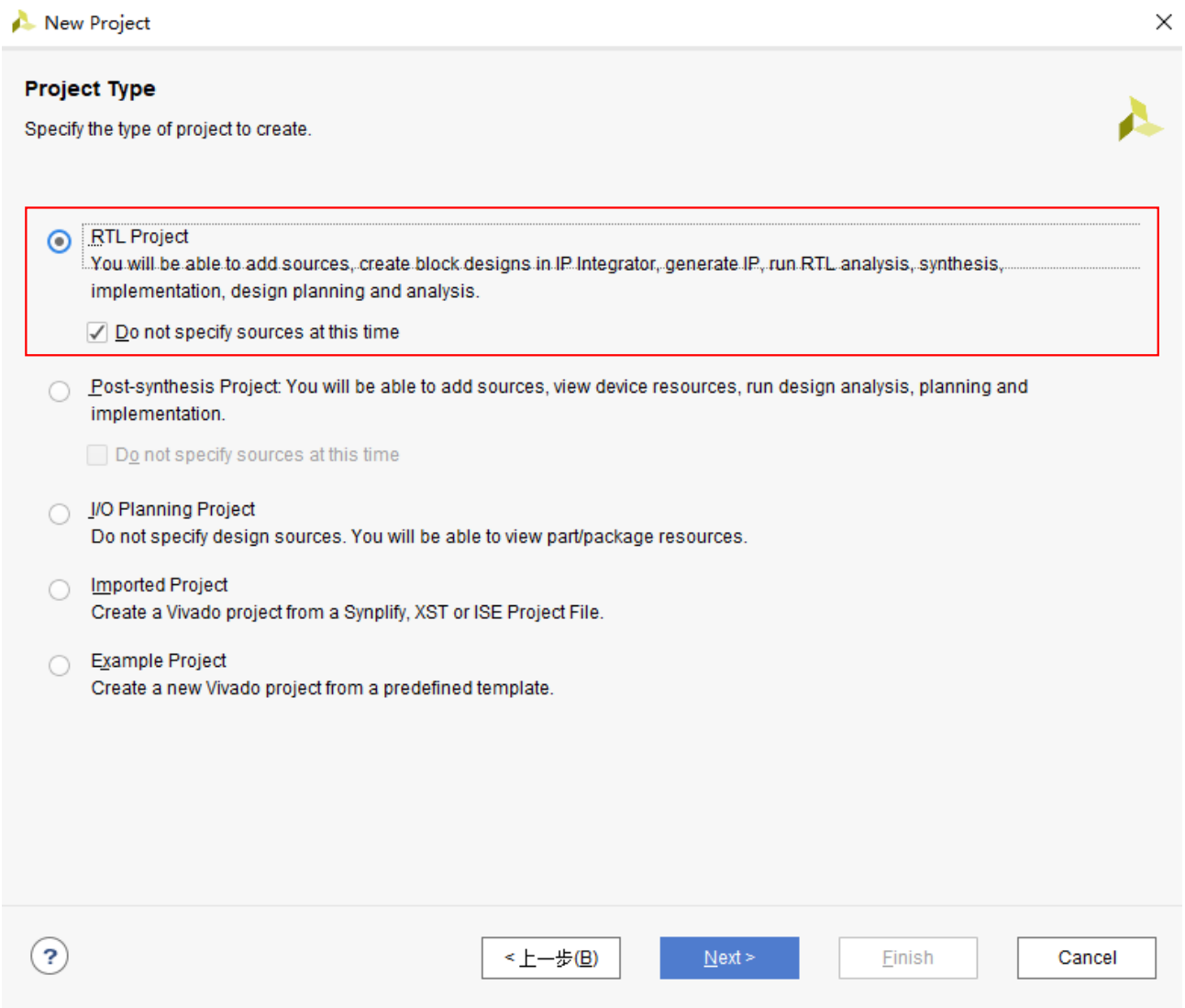


输入项目名称

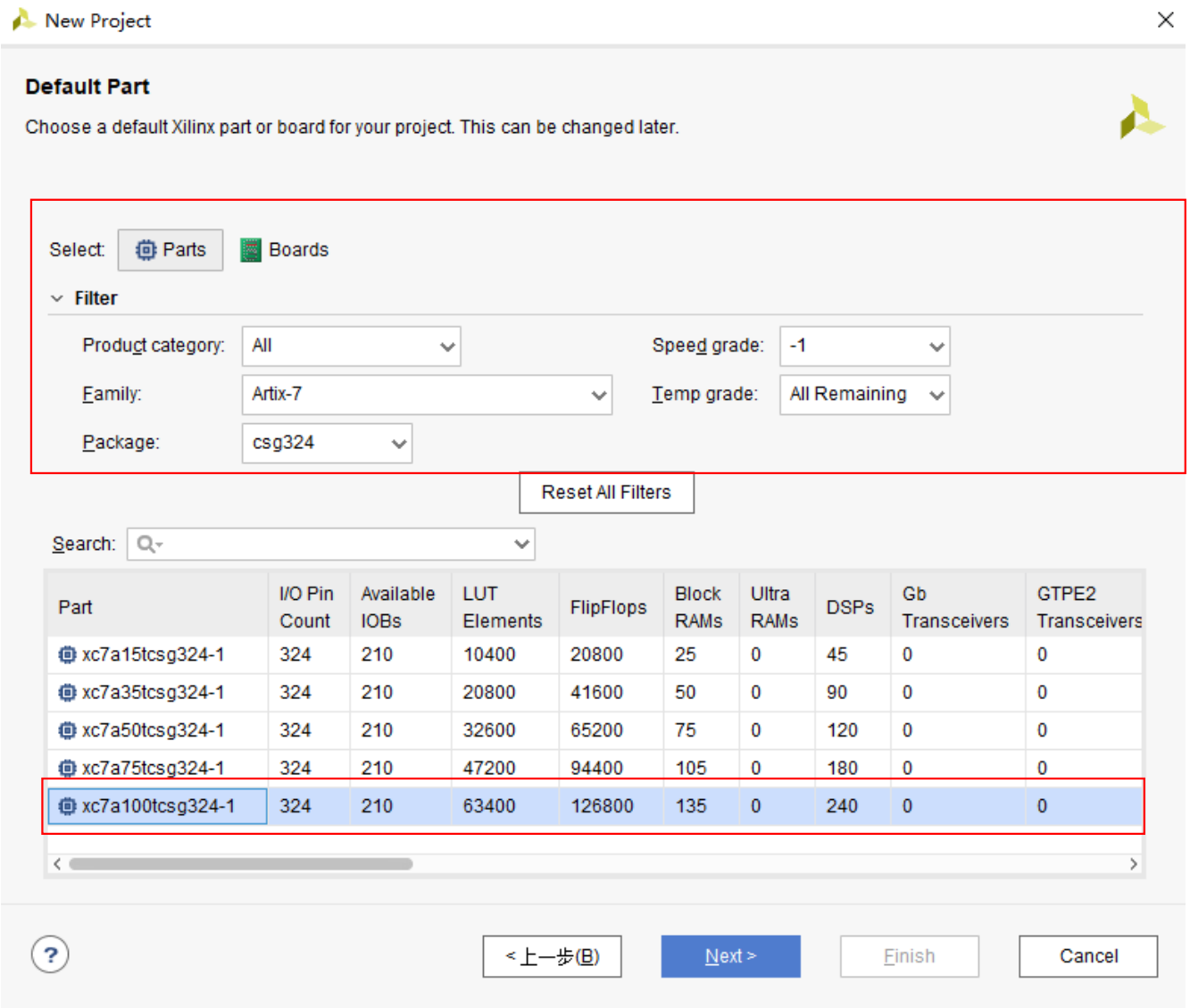
注意工程路径及工程名不要有中文字符

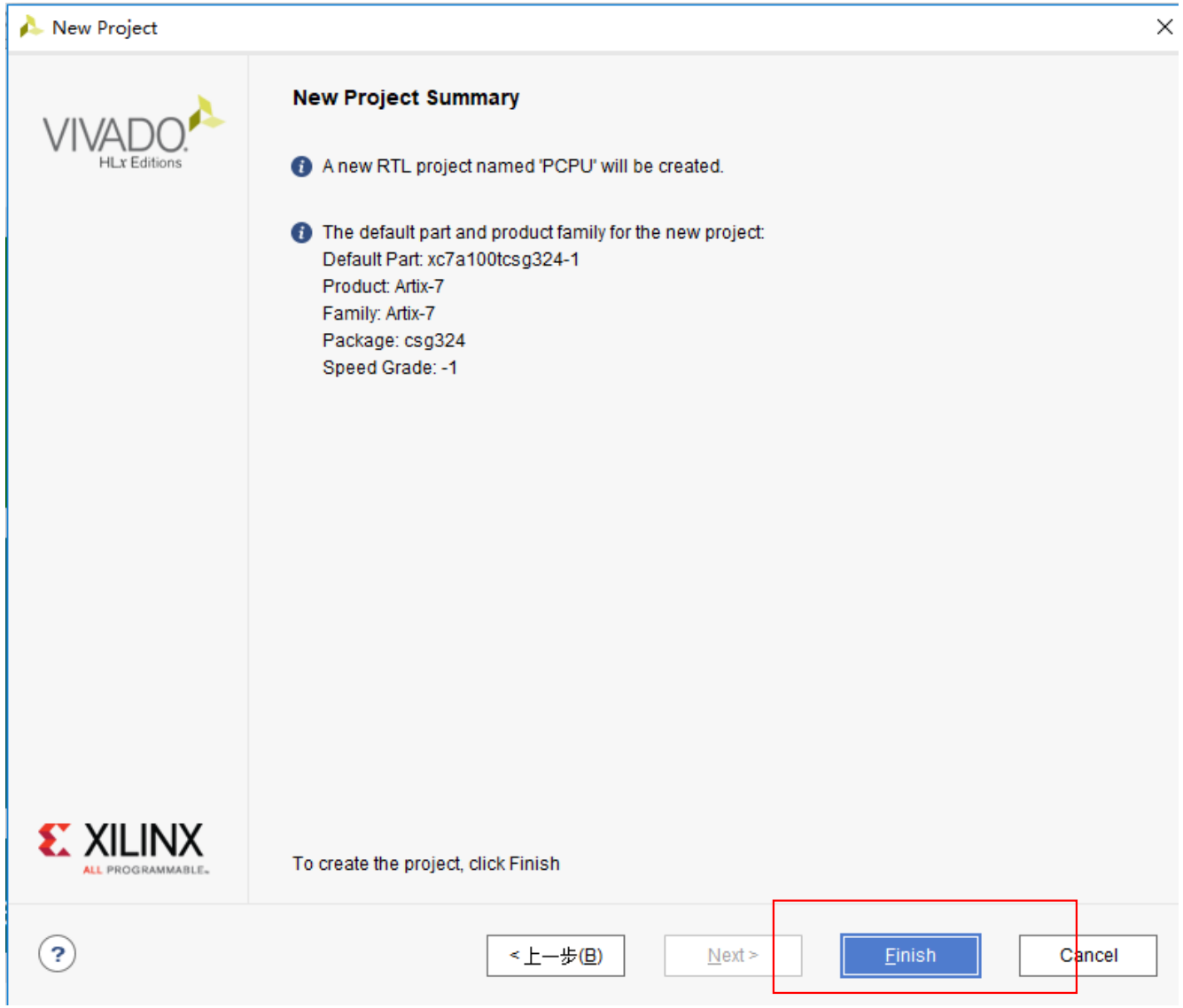


选择项目类型

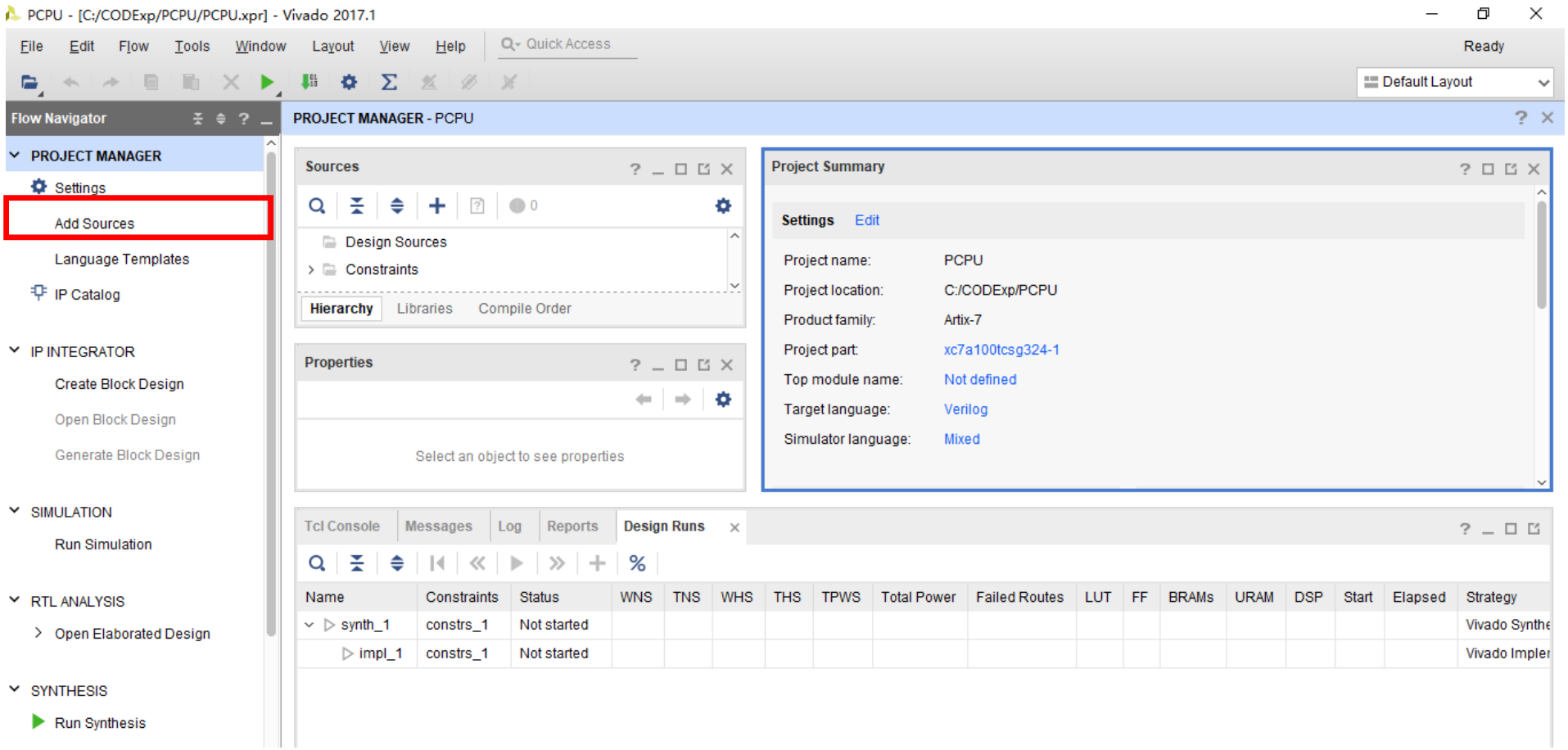

New Project
Project Type
Specify the type of project to create.
RTL Project
You will be able to add sources, create block designs in IP Integrator, generate IP, run RTL analysis, synthesis, implementation, design planning and analysis.
Do not specify sources at this time
Post-synthesis Project: You will be able to add sources, view device resources, run design analysis, planning and implementation.
Do not specify sources at this time
I/O Planning Project
Do not specify design sources. You will be able to view part/package resources.
Imported Project
Create a Vivado project from a Synplify, XST or ISE Project File.
Example Project
Create a new Vivado project from a predefined template.
< 上一步(B)
Next >
Finish
Cancel

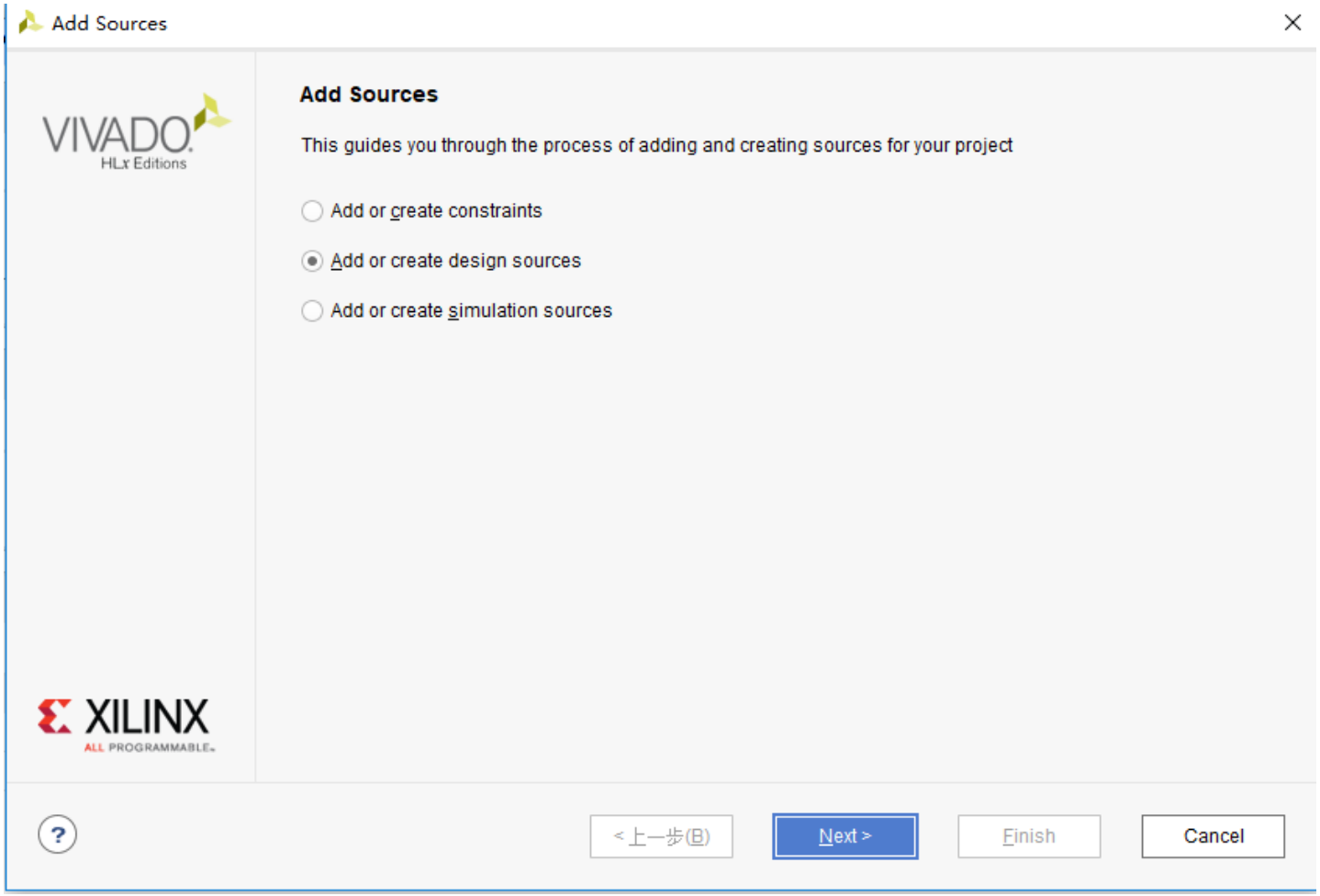
选择FPGA型号

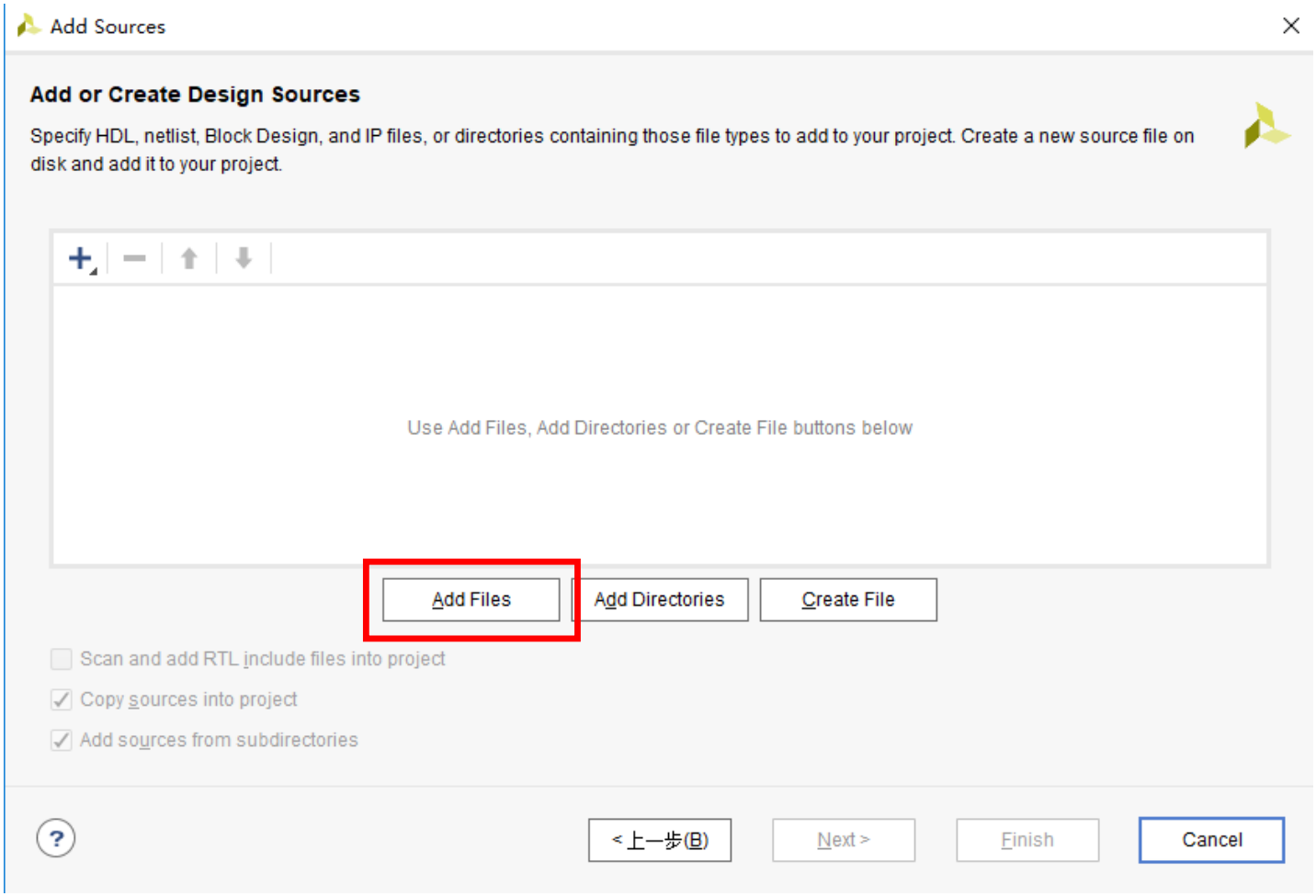

New Project
Default Part
Choose a default Xilinx part or board for your project. This can be changed later.
Select: Parts Boards
Filter
Product category: All
Speed grade: -1
Family: Artix-7
Temp grade: All Remaining
Package: csg324
Reset All Filters
Search:
Part | I/O Pin Count | Available IOBs | LUT Elements | FlipFlops | Block RAMs | Ultra RAMs | DSPs | Gb Transceivers | GTPE2 Transceivers
xc7a15tcsg324-1 | 324 | 210 | 10400 | 20800 | 25 | 0 | 45 | 0 | 0
xc7a35tcsg324-1 | 324 | 210 | 20800 | 41600 | 50 | 0 | 90 | 0 | 0
xc7a50tcsg324-1 | 324 | 210 | 32600 | 65200 | 75 | 0 | 120 | 0 | 0
xc7a75tcsg324-1 | 324 | 210 | 47200 | 94400 | 105 | 0 | 180 | 0 | 0
xc7a100tcsg324-1 | 324 | 210 | 63400 | 126800 | 135 | 0 | 240 | 0 | 0
< 上一步(B)
Next >
Finish
Cancel


New Project
VIVADO
HLx Editions
New Project Summary
A new RTL project named 'PCPU' will be created.
The default part and product family for the new project:
Default Part: xc7a100tcsg324-1
Product: Artix-7
Family: Artix-7
Package: csg324
Speed Grade: -1
XILINX
ALL PROGRAMMABLE.
To create the project, click Finish
< 上一步(B)
Next >
Finish
Cancel

项目管理器——添加文件


PCPU - [C:/CODExp/PCPU/PCPU.xpr] - Vivado 2017.1
File Edit Flow Tools Window Layout View Help
Quick Access
Ready
Default Layout
Flow Navigator
PROJECT MANAGER - PCPU
PROJECT MANAGER
Settings
Add Sources
Language Templates
IP Catalog
IP INTEGRATOR
Create Block Design
Open Block Design
Generate Block Design
SIMULATION
Run Simulation
RTL ANALYSIS
Open Elaborated Design
SYNTHESIS
Run Synthesis
Sources
Design Sources
Constraints
Hierarchy Libraries Compile Order
Properties
Select an object to see properties
Project Summary
Settings Edit
Project name: PCPU
Project location: C:/CODExp/PCPU
Product family: Artix-7
Project part: xc7a100tcsg324-1
Top module name: Not defined
Target language: Verilog
Simulator language: Mixed
Tcl Console Messages Log Reports Design Runs
Name Constraints Status WNS TNS WHS THS TPWS Total Power Failed Routes LUT FF BRAMs URAM DSP Start Elapsed Strategy
synth_1 constrs_1 Not started Vivado Synthe
impl_1 constrs_1 Not started Vivado Impler


Add Sources
VIVADO
HLx Editions
Add Sources
This guides you through the process of adding and creating sources for your project
Add or create constraints
Add or create design sources
Add or create simulation sources
XILINX
ALL PROGRAMMABLE.
< 上一步(B)
Next >
Finish
Cancel



添加给你的代码CPU实现文件

再添加给你的FPGA相关的所有文件

Add Sources

Add or Create Design Sources

Specify HDL, netlist, Block Design, and IP files, or directories containing those file types to add to your project. Create a new source file on disk and add it to your project.

Index	Name	Library	Location
1	EXT.v	xil_defaultlib	C:/work/CODExp2019Spring/toStudent/SCPU
2	NPC.v	xil_defaultlib	C:/work/CODExp2019Spring/toStudent/SCPU
3	PC.v	xil_defaultlib	C:/work/CODExp2019Spring/toStudent/SCPU
4	RF.v	xil_defaultlib	C:/work/CODExp2019Spring/toStudent/SCPU
5	alu.v	xil_defaultlib	C:/work/CODExp2019Spring/toStudent/SCPU
6	ctrl.v	xil_defaultlib	C:/work/CODExp2019Spring/toStudent/SCPU

Add Files | Add Directories | Create File

Scan and add RTL include files into project

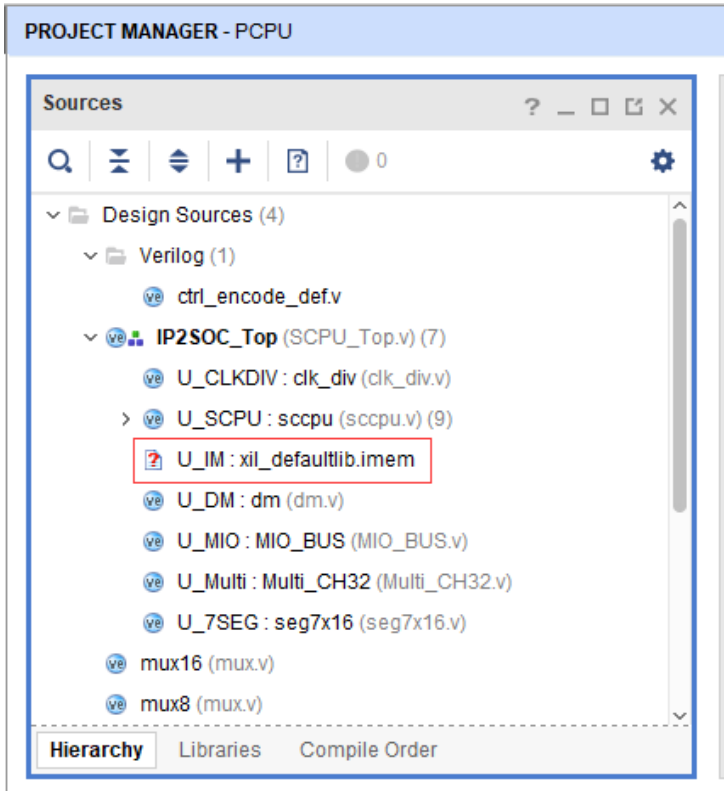
Copy sources into project

Add sources from subdirectories

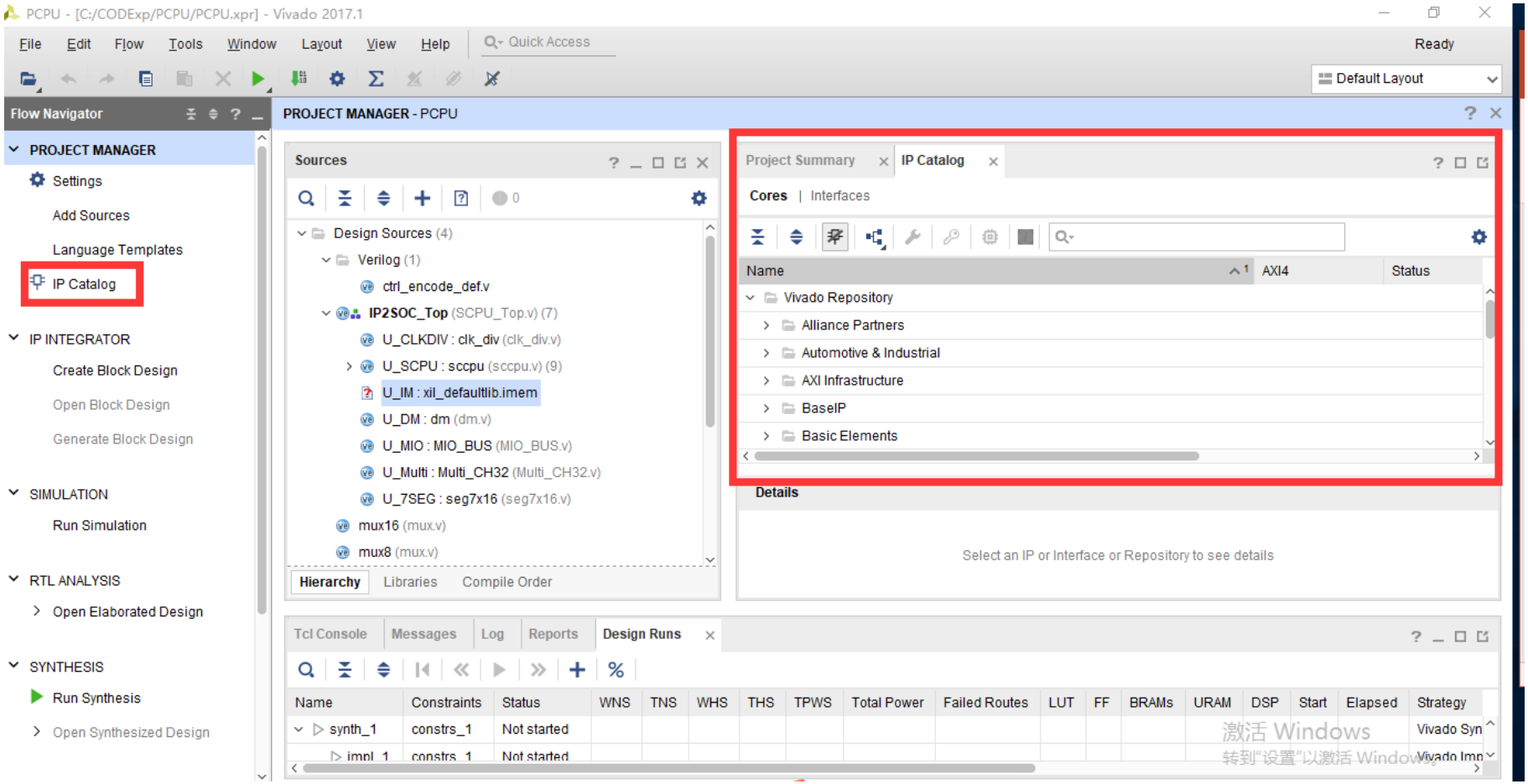
复制文件到在项目目录

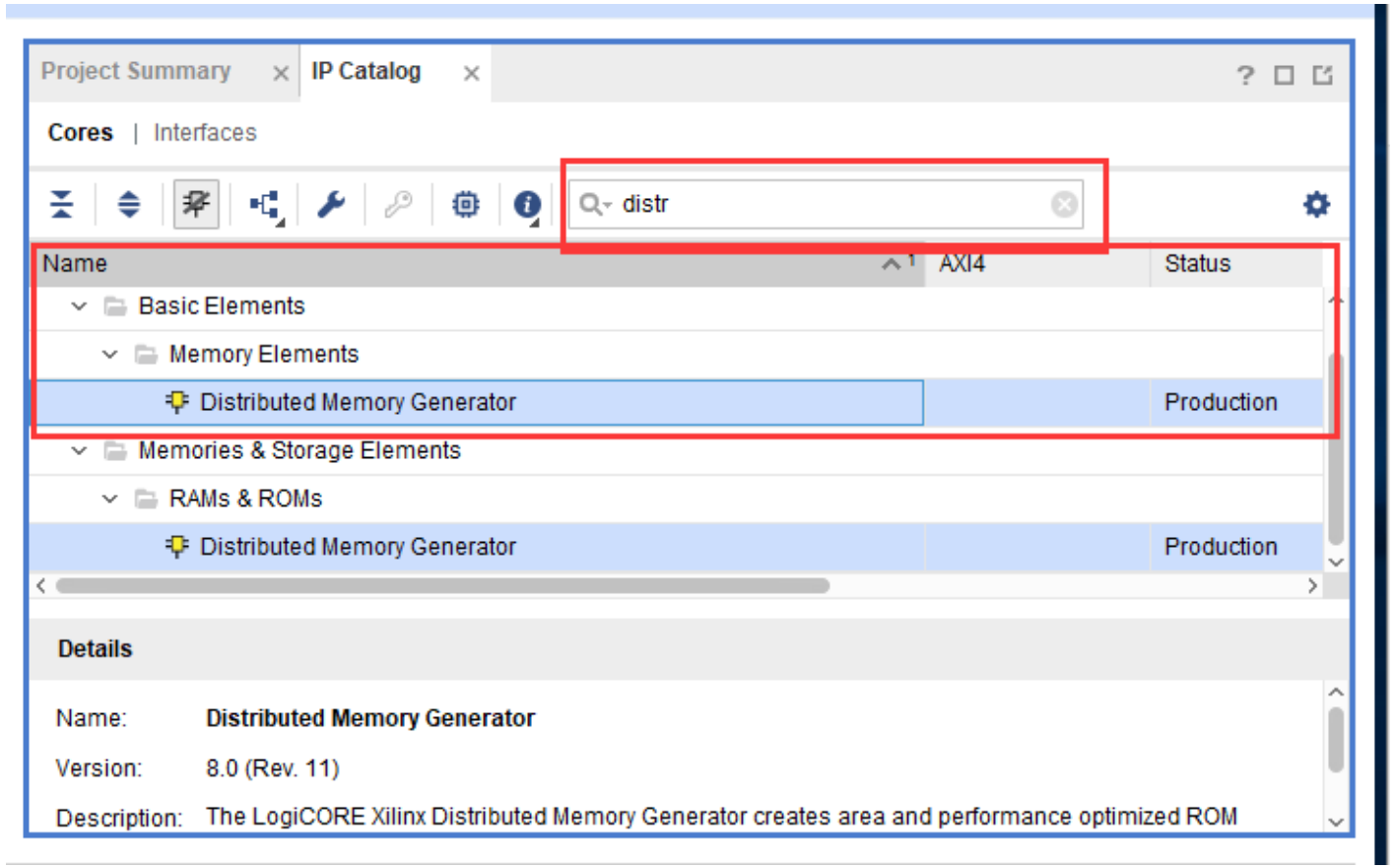
< 上一步(B) | Next > | Finish | Cancel

添加设计文件后的项目管理器



ROM IP核生成


PCPU - [C:/CODExp/PCPU/PCPU.xpr] - Vivado 2017.1
File Edit Flow Tools Window Layout View Help
Quick Access
Ready
Default Layout
Flow Navigator
PROJECT MANAGER
Settings
Add Sources
Language Templates
IP Catalog
IP INTEGRATOR
Create Block Design
Open Block Design
Generate Block Design
SIMULATION
Run Simulation
RTL ANALYSIS
Open Elaborated Design
SYNTHESIS
Run Synthesis
Open Synthesized Design
PROJECT MANAGER - PCPU
Sources
Design Sources (4)
Verilog (1)
ctrl_encode_def.v
IP2SOC_Top (SCPU_Top.v) (7)
U_CLKDIV : clk_div (clk_div.v)
U_SCPU : sccpu (sccpu.v) (9)
U_IM : xil_defaultlib.imem
U_DM : dm (dm.v)
U_MIO : MIO_BUS (MIO_BUS.v)
U_Multi : Multi_CH32 (Multi_CH32.v)
U_7SEG : seg7x16 (seg7x16.v)
mux16 (mux.v)
mux8 (mux.v)
Hierarchy Libraries Compile Order
Project Summary
IP Catalog
Cores | Interfaces
Name AXI4 Status
Vivado Repository
Alliance Partners
Automotive & Industrial
AXI Infrastructure
BaseIP
Basic Elements
Details
Select an IP or Interface or Repository to see details
Tcl Console Messages Log Reports Design Runs
Name Constraints Status WNS TNS WHS THS TPWS Total Power Failed Routes LUT FF BRAMs URAM DSP Start Elapsed Strategy
synth_1 constrs_1 Not started Vivado Syn
impl_1 constrs_1 Not started Vivado Imp
激活 Windows
转到"设置"以激活 Windows。



双击“Distributed Memory Generator”

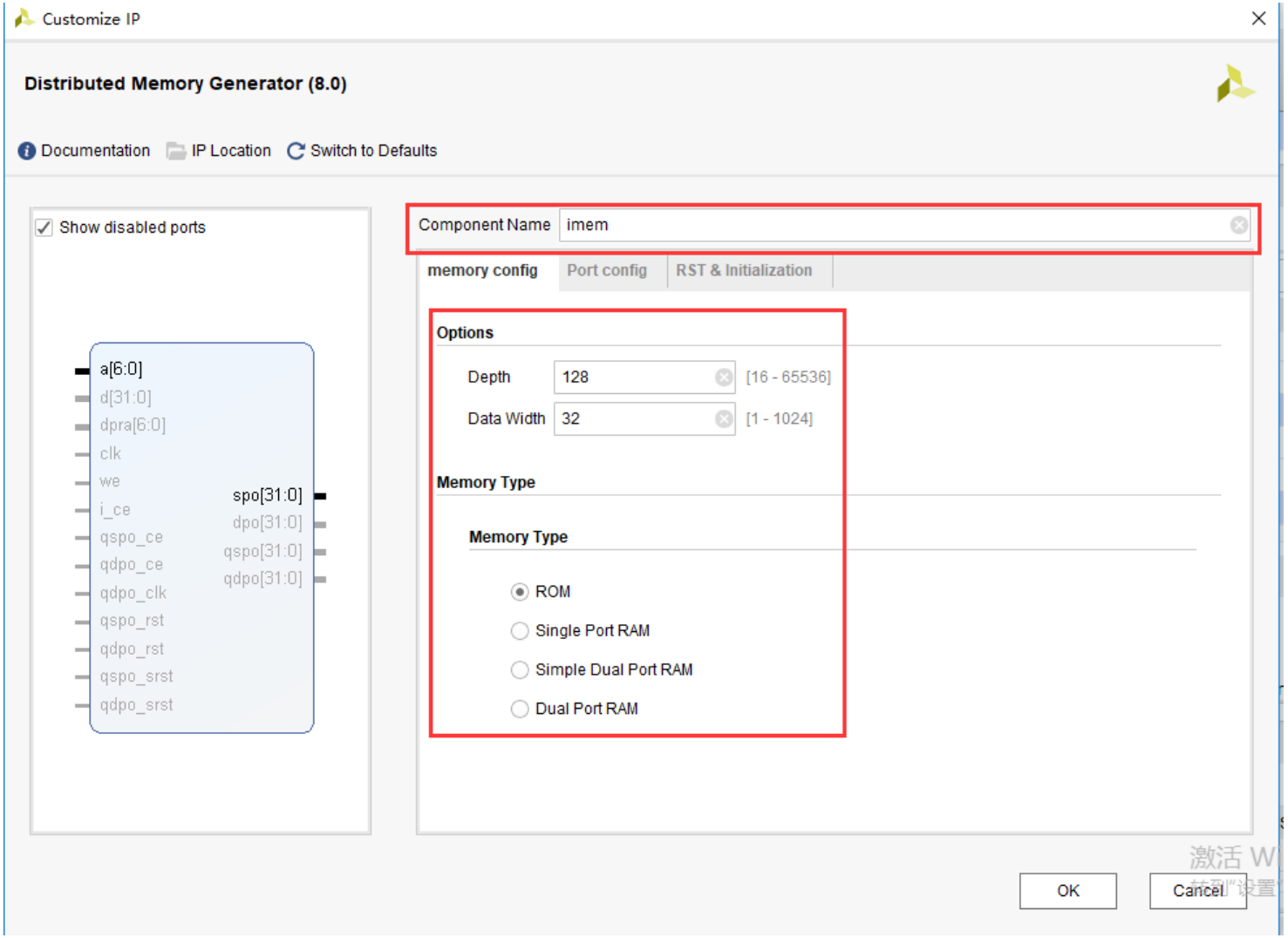
工程实践中常用的RAM是“同步读RAM”（第一拍发读请求和读地址，第二拍RAM才会输出读出的数据），对应Xilinx FPGA中的block RAM;

这里我们采用“**异步读RAM**”（读时序类似于寄存器堆的读，当拍给地址、出数据，其写时序行为和同步读RAM一样），对应Xilinx FPGA中的distributed RAM。

配置ROM IP核 - memory config

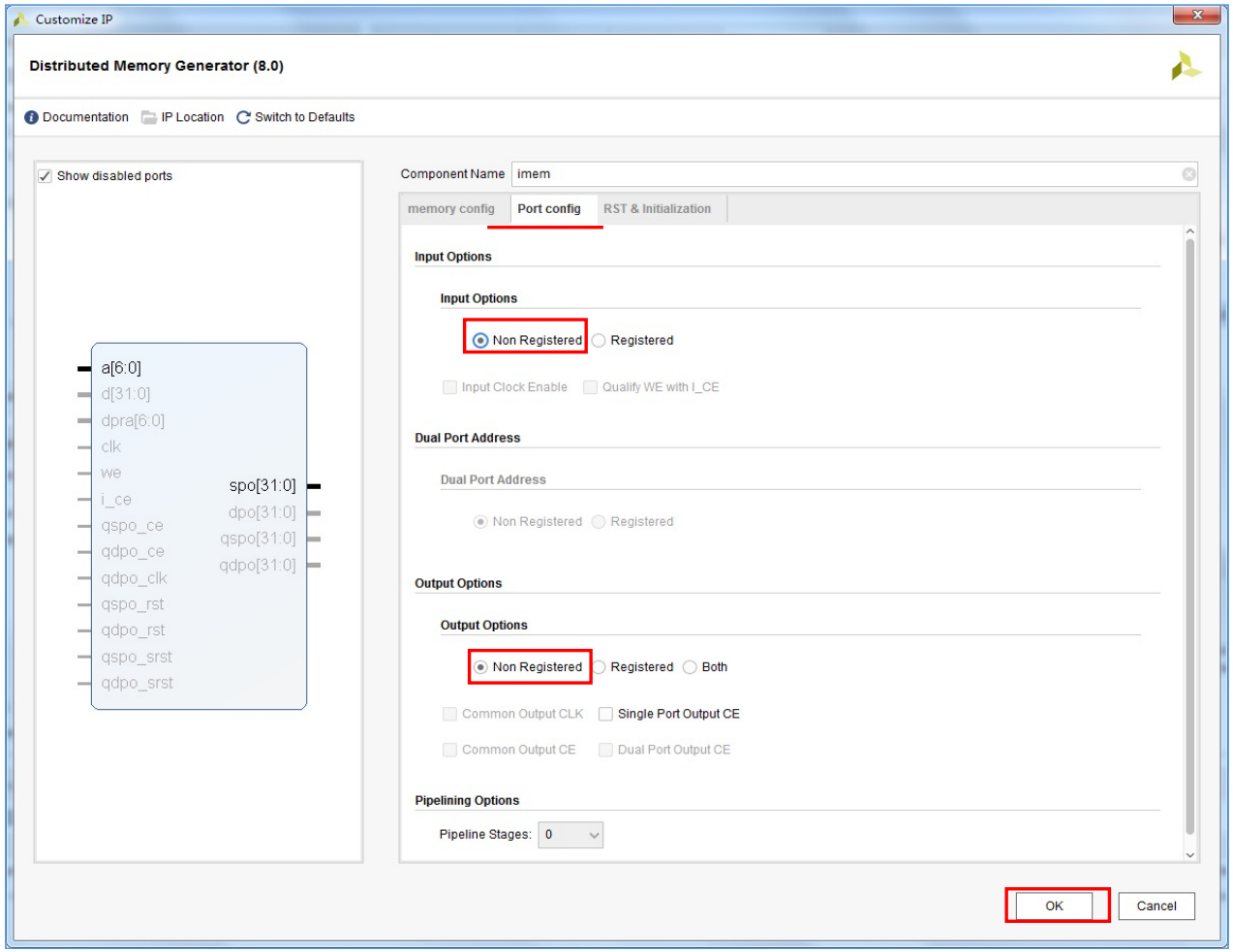
Component Name 必须为imem

其他选项也按照此处的配置进行选择



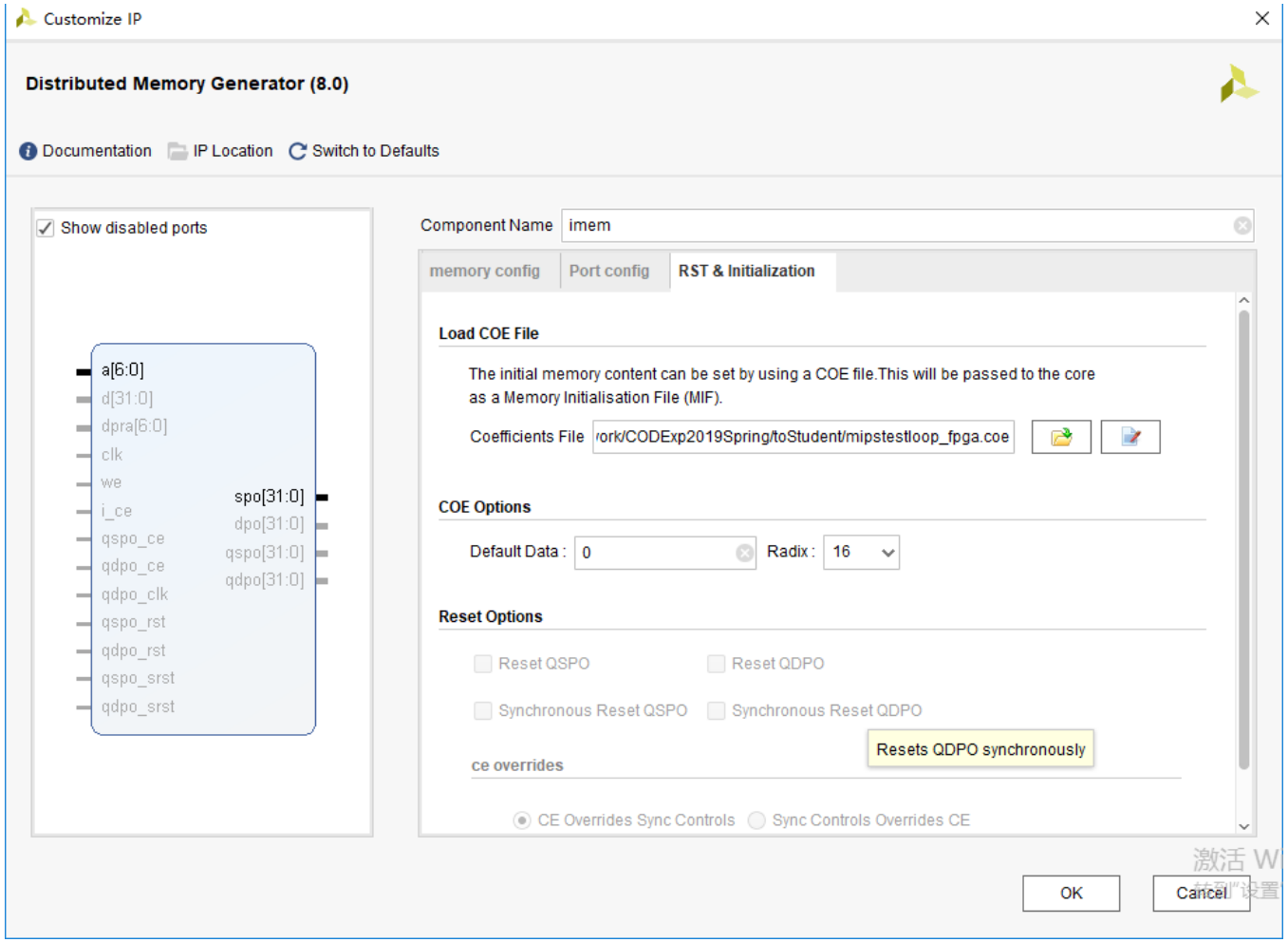
配置ROM IP核 - Port config

选择“Port config”标签



配置ROM IP核 - RST & Initialization

- 选择“RST&Initialization”标签
- “Load COE File”处选择给你的coe目录下的riscv-studentnosorting.coe
- COE(Coefficient)文件是ROM的初始化需要使用来传递参数

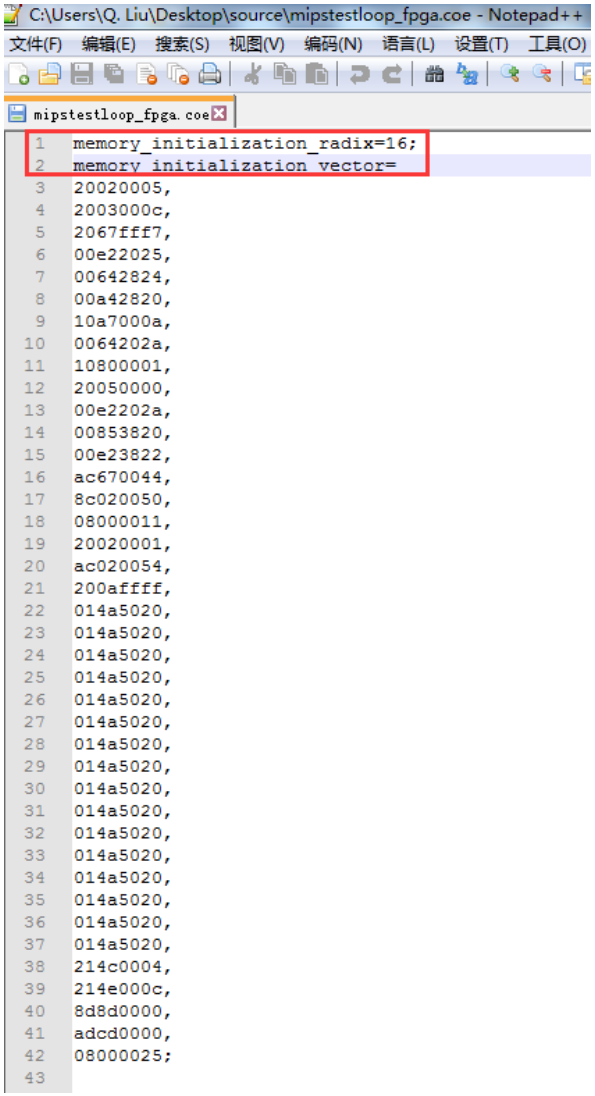


COE文件

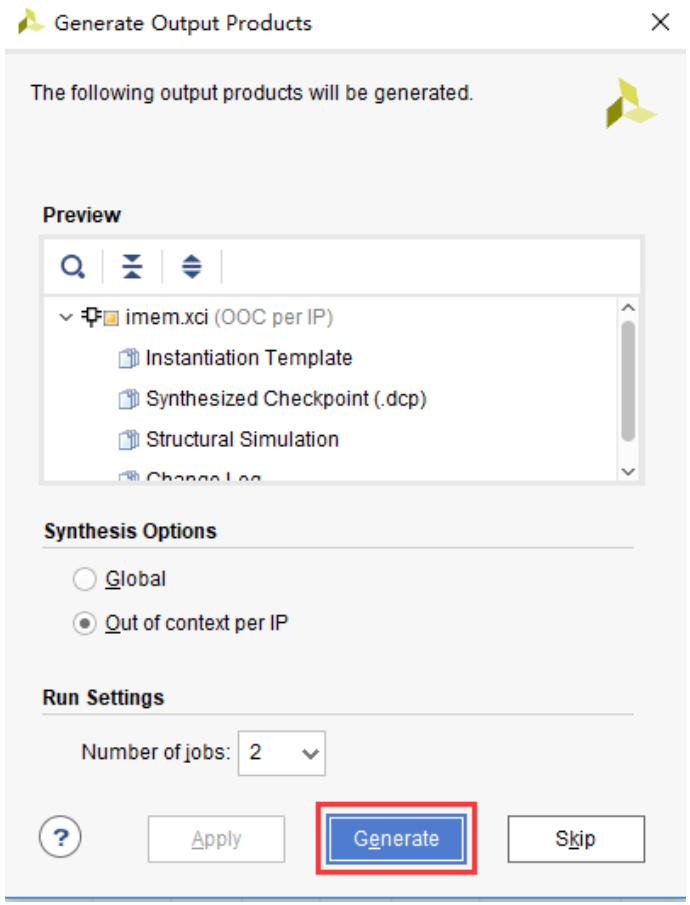
1. 写汇编代码
2. Venus生成机器代码
3. prep-machine-code.py去除开头的0x
4. prep-coe-code.py将之变为coe文件：
   - 在机器代码的开头加上两行

     memory_initialization_radix=16;

     memory_initialization_vector=
   - 其余各行末尾添加“,”
   - 最后一行末尾添加“;”

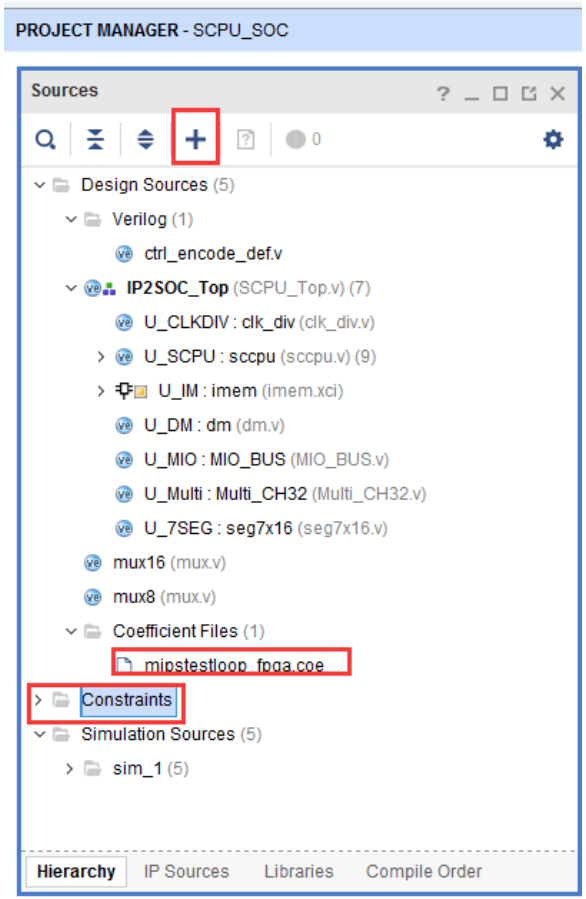


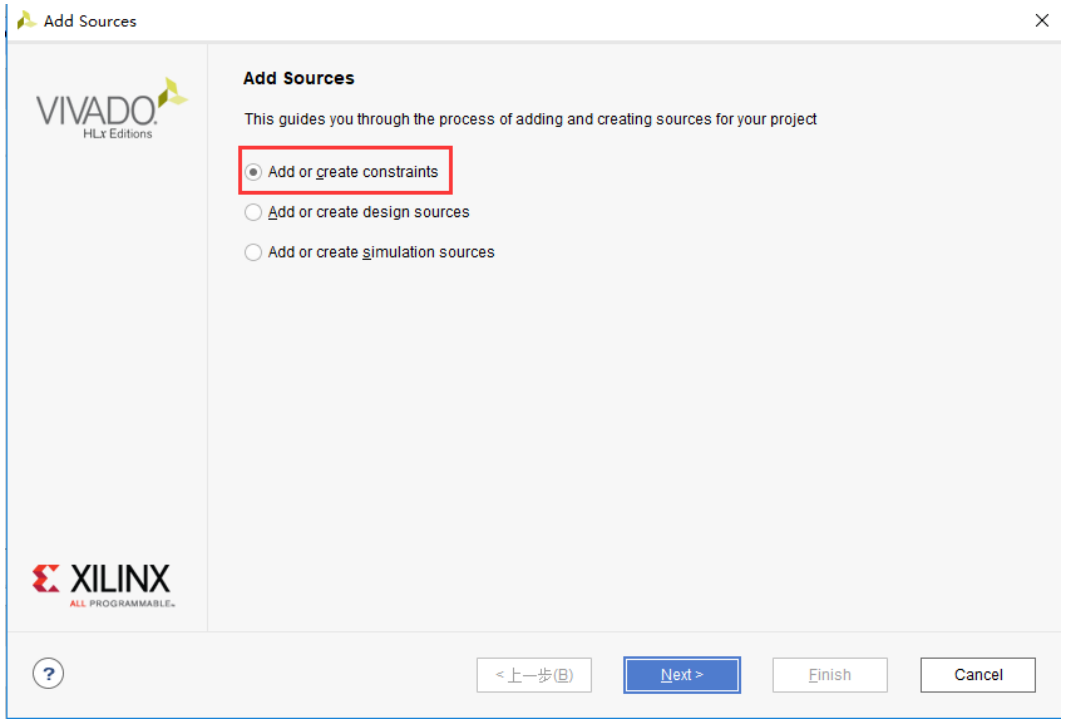
产生ROM IP核

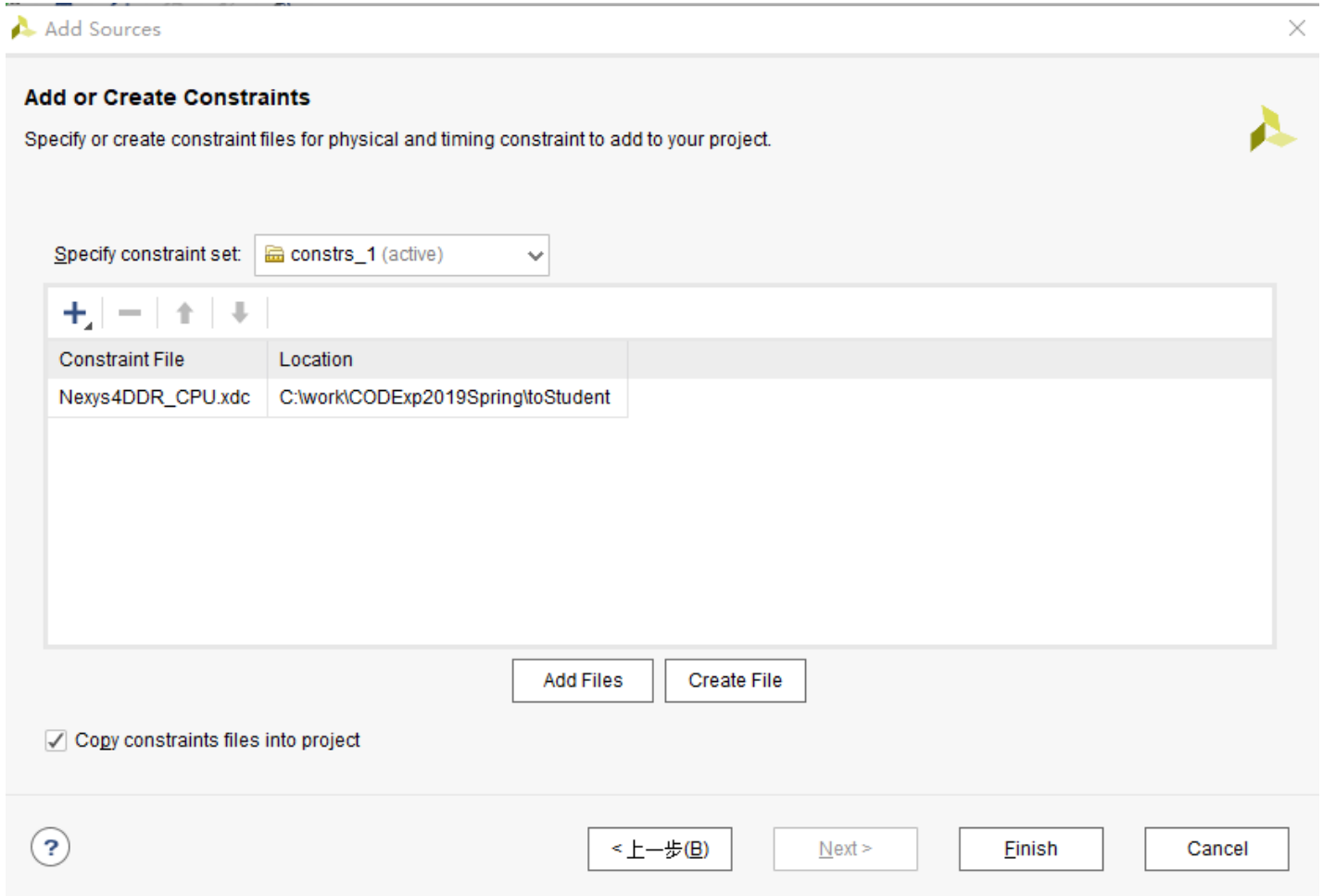

Generate Output Products
The following output products will be generated.
Preview
imem.xci (OOC per IP)
Instantiation Template
Synthesized Checkpoint (.dcp)
Structural Simulation
Synthesis Options
Global
Out of context per IP
Run Settings
Number of jobs: 2
Apply
Generate
Skip

添加约束文件

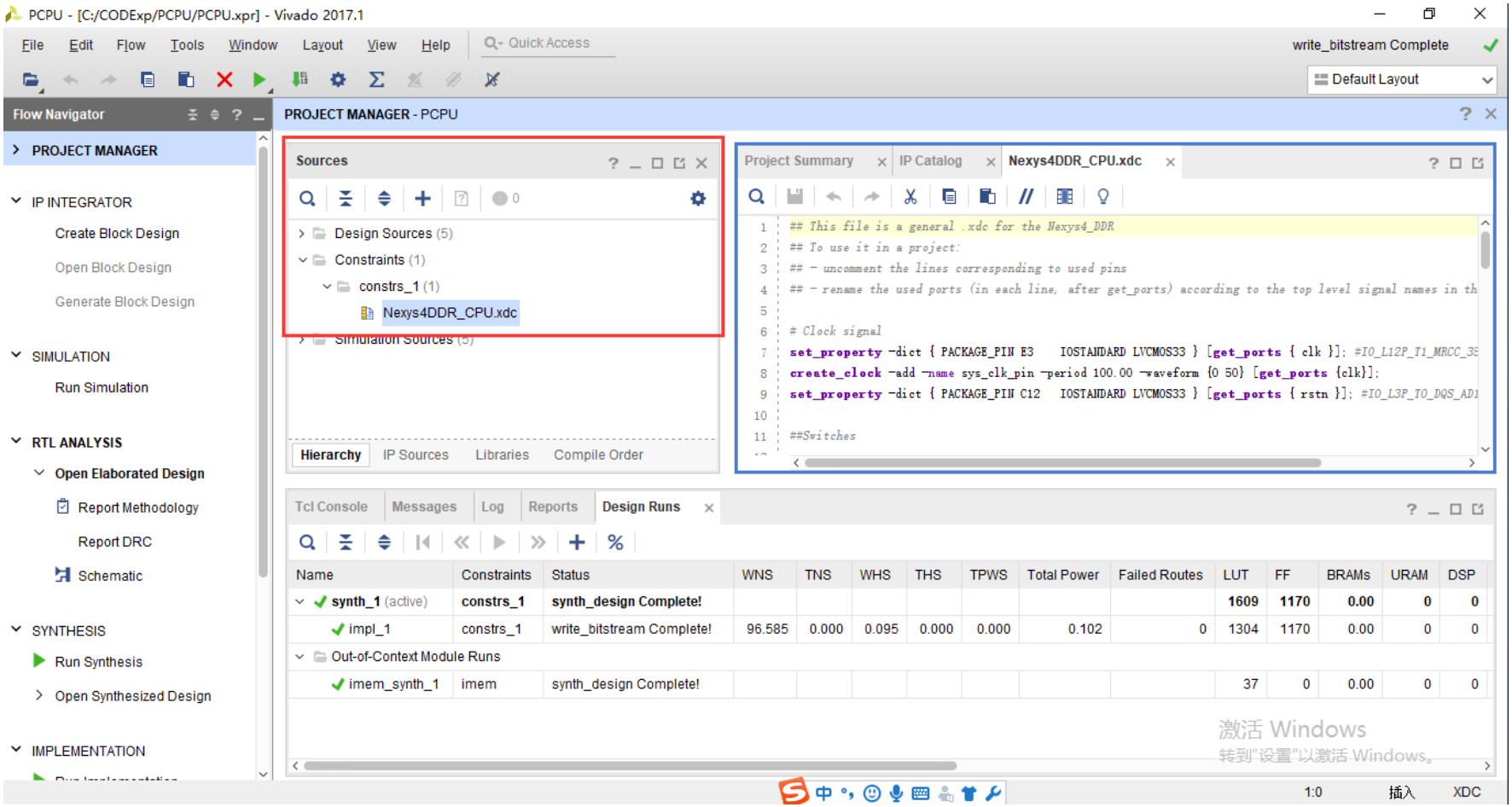
Flow Navigator->Project Manager->Add Sources







添加给你的代码目录下的约束文件Nexys4DDR_CPU.xdc



Project Manager->Sources窗口中

双击Constraints下constrs_1下Nexys4DDR_CPU.xdc

FPGA板上的各个部件特性可参考发给你们的Nexys4-DDR_rm.pdf

约束文件

- **约束文件**描述顶层电路的input/output与FPGA芯片的I/O引脚之间的绑定关系

```
##7 segment display
set_property -dict  PACKAGE_PIN T10   IOSTANDARD LVCMOS33 } [get_ports { disp_seg_o[0] }]; #IO_L24N_T3_A00_D16_14 Sch=ca
set_property -dict  PACKAGE_PIN R10   IOSTANDARD LVCMOS33 } [get_ports { disp_seg_o[1] }]; #IO_25_14 Sch=cb
set_property -dict  PACKAGE_PIN K16   IOSTANDARD LVCMOS33 } [get_ports { disp_seg_o[2] }]; #IO_25_15 Sch=cc
set_property -dict  PACKAGE_PIN K13   IOSTANDARD LVCMOS33 } [get_ports { disp_seg_o[3] }]; #IO_L7P_T2_A26_15 Sch=cd
set_property -dict  PACKAGE_PIN P15   IOSTANDARD LVCMOS33 } [get_ports { disp_seg_o[4] }]; #IO_L13P_T2_MRCC_14 Sch=ce
set_property -dict  PACKAGE_PIN T11   IOSTANDARD LVCMOS33 } [get_ports { disp_seg_o[5] }]; #IO_L19P_T3_A10_D26_14 Sch=cf
set_property -dict  PACKAGE_PIN L18   IOSTANDARD LVCMOS33 } [get_ports { disp_seg_o[6] }]; #IO_L4P_T0_D04_14 Sch=cg
set_property -dict  PACKAGE_PIN H15   IOSTANDARD LVCMOS33 } [get_ports { disp_seg_o[7] }]; #IO_L19N_T3_A21_VREF_15 Sch=dp

set_property -dict  PACKAGE_PIN J17   IOSTANDARD LVCMOS33 } [get_ports { disp_an_o[0] }]  #IO_L23P_T3_FOE_B_15 Sch=an[0]
set_property -dict  PACKAGE_PIN J18   IOSTANDARD LVCMOS33 } [get_ports { disp_an_o[1] }]  #IO_L23N_T3_FWE_B_15 Sch=an[1]
set_property -dict  PACKAGE_PIN T9    IOSTANDARD LVCMOS33 } [get_ports { disp_an_o[2] }]  #IO_L24P_T3_A01_D17_14 Sch=an[2
set_property -dict  PACKAGE_PIN J14   IOSTANDARD LVCMOS33 } [get_ports { disp_an_o[3] }]  #IO_L19P_T3_A22_15 Sch=an[3]
set_property -dict  PACKAGE_PIN P14   IOSTANDARD LVCMOS33 } [get_ports { disp_an_o[4] }]  #IO_L8N_T1_D12_14 Sch=an[4]
set_property -dict  PACKAGE_PIN T14   IOSTANDARD LVCMOS33 } [get_ports { disp_an_o[5] }]  #IO_L14P_T2_SRCC_14 Sch=an[5]
set_property -dict  PACKAGE_PIN K2    IOSTANDARD LVCMOS33 } [get_ports { disp_an_o[6] }]  #IO_L23P_T3_35 Sch=an[6]
set_property -dict  PACKAGE_PIN U13   IOSTANDARD LVCMOS33 } [get_ports { disp_an_o[7] }]  #IO_L23N_T3_A02_D18_14 Sch=an[7
```

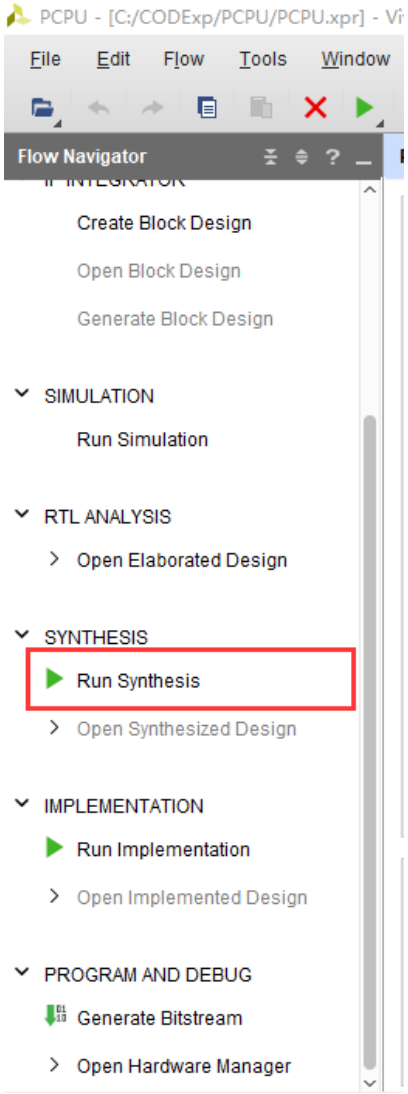
```
# Clock signal
set_property -dict { PACKAGE_PIN E3    IOSTANDARD LVCMOS33 } [get_ports { clk }]; #IO_L12P_T1_MRCC_35 Sch=clk100mhz
create_clock -add -name sys_clk_pin -period 100.00 -waveform {0 50} [get_ports {clk}]
set_property -dict { PACKAGE_PIN C12   IOSTANDARD LVCMOS33 } [get_ports { rstn }]; #IO_L3P_T0_DQS_AD1P_15 Sch=cpu_resetn

##Switches

set_property -dict { PACKAGE_PIN J15   IOSTANDARD LVCMOS33 } [get_ports { sw_i[0] }]; #IO_L24N_T3_RS0_15 Sch=sw[0]
set_property -dict { PACKAGE_PIN L16   IOSTANDARD LVCMOS33 } [get_ports { sw_i[1] }]; #IO_L3N_T0_DQS_EMCCLK_14 Sch=sw[1]
set_property -dict { PACKAGE_PIN M13   IOSTANDARD LVCMOS33 } [get_ports { sw_i[2] }]; #IO_L6N_T0_D08_VREF_14 Sch=sw[2]
set_property -dict { PACKAGE_PIN R15   IOSTANDARD LVCMOS33 } [get_ports { sw_i[3] }]; #IO_L13N_T2_MRCC_14 Sch=sw[3]
set_property -dict { PACKAGE_PIN R17   IOSTANDARD LVCMOS33 } [get_ports { sw_i[4] }]; #IO_L12N_T1_MRCC_14 Sch=sw[4]
set_property -dict { PACKAGE_PIN T18   IOSTANDARD LVCMOS33 } [get_ports { sw_i[5] }]; #IO_L7N_T1_D10_14 Sch=sw[5]
set_property -dict { PACKAGE_PIN U18   IOSTANDARD LVCMOS33 } [get_ports { sw_i[6] }]; #IO_L17N_T2_A13_D29_14 Sch=sw[6]
set_property -dict { PACKAGE_PIN R13   IOSTANDARD LVCMOS33 } [get_ports { sw_i[7] }]; #IO_L5N_T0_D07_14 Sch=sw[7]
set_property -dict { PACKAGE_PIN T8    IOSTANDARD LVCMOS18 } [get_ports { sw_i[8] }]; #IO_L24N_T3_34 Sch=sw[8]
set_property -dict { PACKAGE_PIN U8    IOSTANDARD LVCMOS18 } [get_ports { sw_i[9] }]; #IO_25_34 Sch=sw[9]
set_property -dict { PACKAGE_PIN R16   IOSTANDARD LVCMOS33 } [get_ports { sw_i[10] }]; #IO_L15P_T2_DQS_RDWR_B_14 Sch=sw[10]
set_property -dict { PACKAGE_PIN T13   IOSTANDARD LVCMOS33 } [get_ports { sw_i[11] }]; #IO_L23P_T3_A03_D19_14 Sch=sw[11]
set_property -dict { PACKAGE_PIN H6    IOSTANDARD LVCMOS33 } [get_ports { sw_i[12] }]; #IO_L24P_T3_35 Sch=sw[12]
set_property -dict { PACKAGE_PIN U12   IOSTANDARD LVCMOS33 } [get_ports { sw_i[13] }]; #IO_L20P_T3_A08_D24_14 Sch=sw[13]
set_property -dict { PACKAGE_PIN U11   IOSTANDARD LVCMOS33 } [get_ports { sw_i[14] }]; #IO_L19N_T3_A09_D25_VREF_14 Sch=sw[14]
set_property -dict { PACKAGE_PIN V10   IOSTANDARD LVCMOS33 } [get_ports { sw_i[15] }]; #IO_L21P_T3_DQS_14 Sch=sw[15]
```

综合（Synthesis）

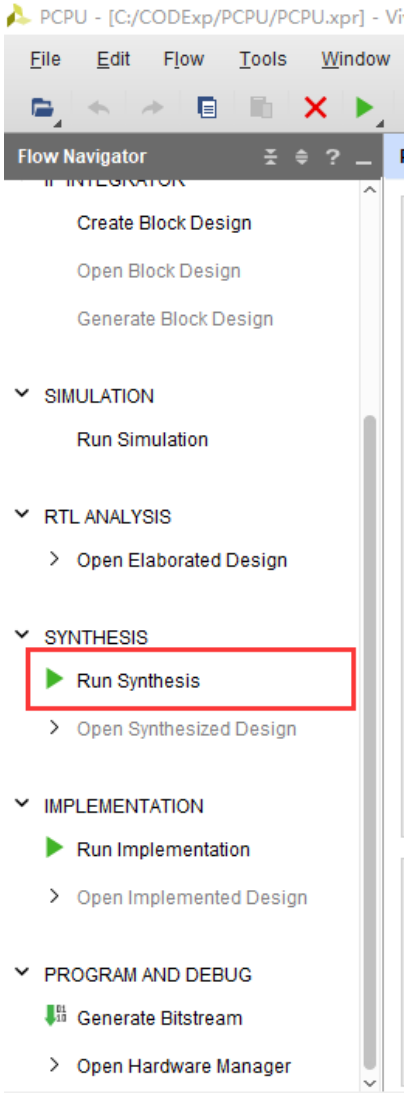
综合是将HDL语言等设计输入翻译成由与、或、非门和RAM、触发器等基本逻辑单元组成的逻辑连接（网表），并根据目标和要求（约束条件）优化生成的RTL层连接。

点击Run Synthesis

然后点OK



点击Run Synthesis
然后点OK



综合所需时间可能较长，注意观察窗口右上角的状态变化

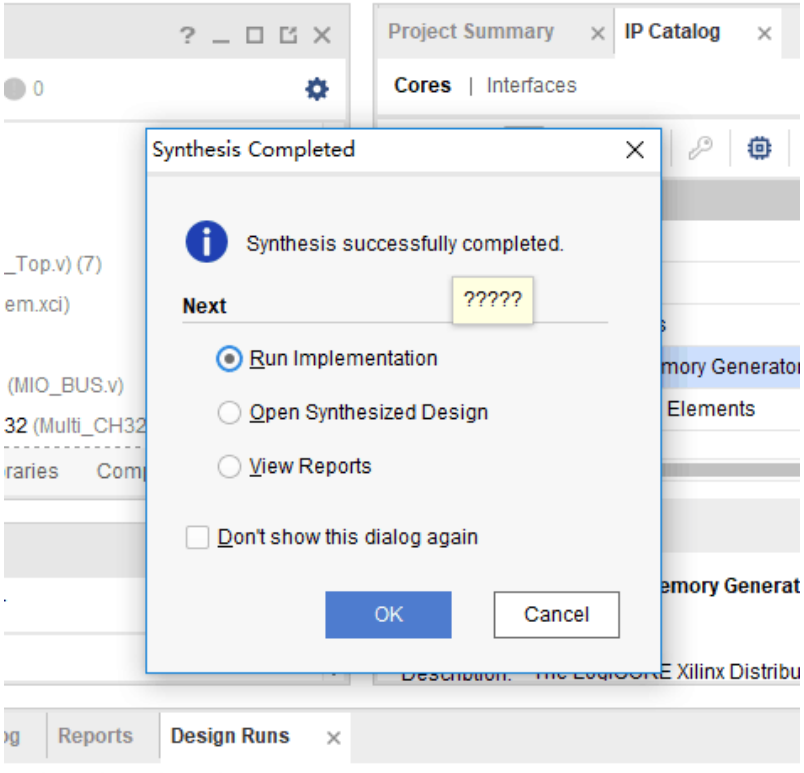
实现（Implementation）

综合完成后会出现如上窗口

实现是综合输出的逻辑网表翻译成所选器件的低层模块和硬件原语，将设计映射到器件结构上，进行布局布线，达到在所选器件上实现设计的目的。

点击Run Implementation，也可以选择Cancel后在左侧Flow Navigator中选择

同样的，这个过程也有可能比较慢，注意观察窗口右上角状态变化



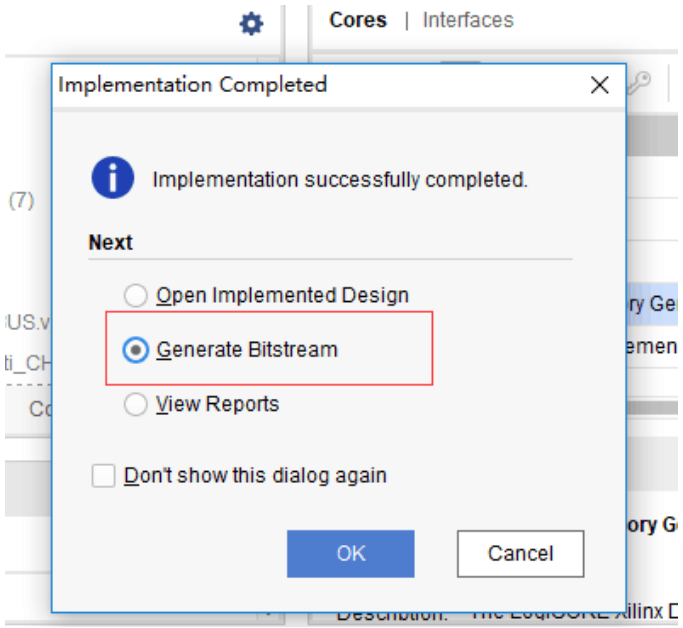
生成bit文件(Generate Bitstream)

实现完成后会出现如上窗口

选择Generate Bitstream，点击Ok

也可以选择Cancel后在左侧Flow Navigator中选择

若在综合和实现之前点击Generate Bitstream会先执行综合、实现，再生成比特流文件。



同样的，这个过程也有可能比较慢，注意观察窗口右上角状态变化

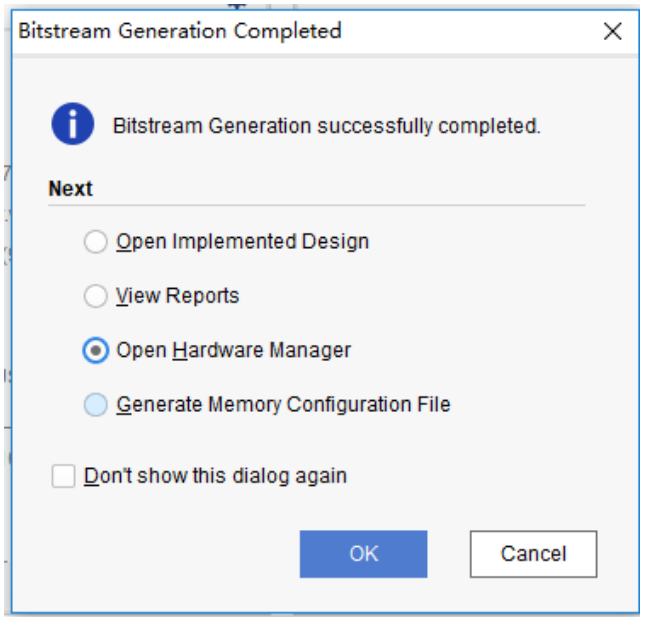
Open Target

位流生成后会出现如上窗口

选择Open Hardware Manager，点击Ok

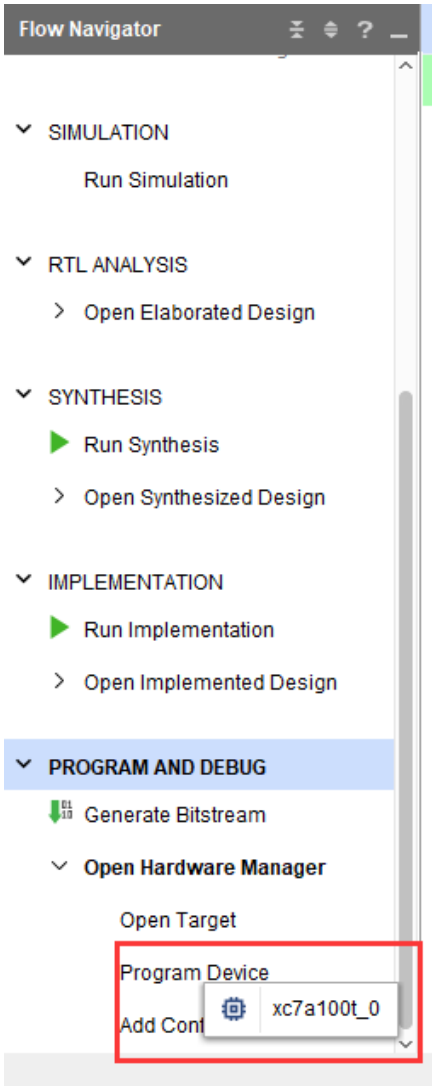
也可以选择Cancel后在左侧Flow Navigator中选择

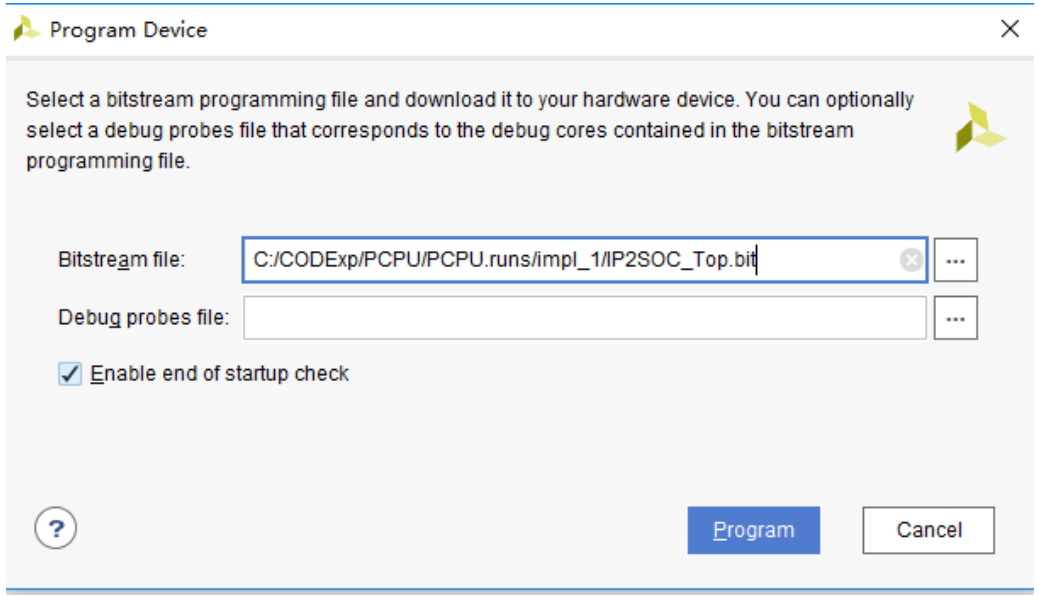
同样的，这个过程也有可能比较慢，注意观察窗口右上角状态变化



下载（Program）

- 如果此前没有连接硬件，则在Open Hardware Manager中选择Open Target
- 选择Auto Connect
- 注意：这时要物理上将设备连接到你的计算机，并打开开关
- 然后再点击Program Device->xc7a100t_0




Program Device
Select a bitstream programming file and download it to your hardware device. You can optionally select a debug probes file that corresponds to the debug cores contained in the bitstream programming file.
Bitstream file:
C:/CODExp/PCPU/PCPU.runs/impl_1/IP2SOC_Top.bit
Debug probes file:
Enable end of startup check
Program
Cancel